

PayMorph

Complete Product & Engineering Blueprint

Pay in XRP. Settle your way.



Hackathon	Flare Summer Signal
Target	Bounty 1 - Interoperable Asset Products
Network	XRPL Testnet + Flare Coston2
Version	1.0 - 27 July 2026

Implementation specification for a working testnet product. Not a security audit, legal opinion, or promise of production readiness.

How to use this blueprint

This document is written so a coding agent can build the product in controlled phases without inventing missing architecture. It is intentionally opinionated: one stack, one payment protocol, explicit state transitions, testable acceptance criteria, and a defined recovery path.

Recommended Codex workflow

Create an empty Git repository, save this PDF beside it, copy the Master Codex Prompt from Section 30 into Codex, and require one phase per pull request. Do not ask Codex to build the entire system in one unreviewed pass.

Non-negotiable operating rules

- The core demo must use a real XRPL Testnet payment, a real FDC-backed direct FXRP mint on Coston2, and real on-chain settlement. No fake success flags.
- Resolve Flare protocol addresses at runtime through the Contract Registry whenever the protocol exposes them. Do not hardcode FXRP or AssetManagerFXRP.
- Use integer base units only. Never use JavaScript floating-point numbers for XRP, FXRP, USDT0, fees, or percentages.
- Store and verify every external identifier: Xaman payload UUID, XRPL transaction hash, Flare transaction hash, user-operation hash, invoice ID, and payment-attempt ID.
- Every webhook and worker job must be idempotent. A retry must not create a second settlement.
- The customer must sign in with Xaman before the payment payload is built, because the Smart Account sender and nonce depend on the actual XRPL account.
- A payment is complete only after the PayMorphRouter emits PaymentSettled and the receipt is reconstructed from on-chain events.
- USDT0 settlement is enabled only when the configured Coston2 pool exists and has sufficient liquidity for the quoted amount. FXRP settlement remains the guaranteed fallback.

Contents

- 1. Product definition
- 2. Users, jobs, and requirements
- 3. Scope and non-goals
- 4. End-to-end product flows
- 5. System architecture
- 6. Exact cross-chain protocol
- 7. Smart-contract specification
- 8. Application services
- 9. Database schema
- 10. HTTP API specification
- 11. Web routes and page behavior
- 12. UI system and components
- 13. State machines and idempotency
- 14. Pricing, quotes, fees, and amounts
- 15. Xaman integration
- 16. Executor, FDC, and direct mint finalization
- 17. Recovery and failure handling
- 18. Security and threat model
- 19. Testing strategy

- 20. Repository and development environment
- 21. Deployment and configuration
- 22. Observability and operations
- 23. CI/CD and release gates
- 24. Product analytics and privacy
- 25. Demo script and judge experience
- 26. Hackathon submission package
- 27. Mainnet roadmap and PMF path
- 28. Codex implementation plan
- 29. Definition of done
- 30. Master Codex prompt
- Appendices A-I

Executive specification

PayMorph is a cross-chain merchant payment product. A merchant creates an invoice denominated in XRP or USD and chooses settlement in FXRP or USDT0. A customer identifies their XRPL account through Xaman, signs one native XRP payment, and the payment triggers a Flare Smart Account user operation. FXRP is minted on Coston2 and atomically routed through PayMorph contracts to one or more recipients. In USDT0 mode, FXRP is swapped through a configured SparkDEX Uniswap V3 route before distribution.

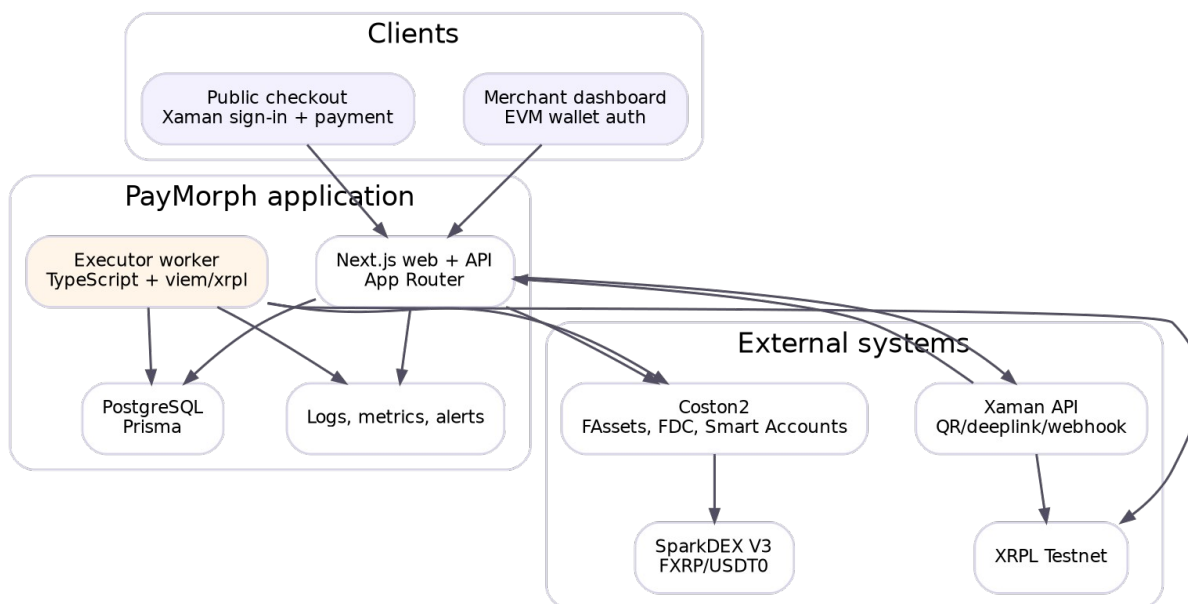


Figure 1. PayMorph high-level architecture

Core promise

One sentence

Customers pay using native XRP from the wallet they already use; merchants receive programmable FXRP or stable-value USDT0 on Flare, with optional automatic revenue splitting.

MVP success condition

A judge can create an invoice, authenticate a payer with Xaman, sign a test XRP payment, watch the XRPL transaction validate, watch the FDC/executor finalization on Coston2, and verify the merchant and optional recipients received the selected settlement asset. Both explorer links appear on a permanent receipt page.

Why Flare is essential

- FAssets represents XRP as FXRP on Flare.
- Flare Smart Accounts let an XRPL payment authorize arbitrary Flare calls without requiring the payer to hold FLR.
- FDC supplies the XRPPayment proof used to finalize direct minting.
- FTSO supplies decentralized XRP/USD data for USD-denominated invoices and quote sanity checks.

- Flare EVM contracts enforce settlement, fees, splits, replay protection, and auditable receipts.

Source anchors: [S1], [S2], [S3], [S4], [S6], [S8]

1. Product definition

1.1 Product name and positioning

Field	Specification
Name	PayMorph
Tagline	Pay in XRP. Settle your way.
Category	Programmable cross-chain checkout and settlement
Primary bounty	Interoperable Asset Products
Initial network	XRPL Testnet + Coston2
Primary initial users	Freelancers, digital merchants, creator platforms, agencies, and small marketplaces
Core differentiation	Native XRP in; merchant-selected Flare asset out; optional automatic splits

1.2 Problem

An XRP holder can send XRP globally, but a merchant may not want XRP price exposure and may need programmable actions such as platform fees, affiliate payouts, or revenue splits. A normal XRPL payment does not directly execute Solidity business logic. Existing bridging interfaces also stop at moving the asset, leaving the merchant to perform conversion and settlement manually.

1.3 Solution

1. Merchant creates a shareable invoice in XRP or USD.
2. Merchant chooses FXRP or USDT0 settlement.
3. Customer identifies their XRPL address using a Xaman SignIn payload.
4. Backend derives the customer personal account and constructs a 0xFE Smart Account user operation.
5. Customer signs one XRP Payment to the FAssets direct-minting Core Vault.
6. Executor obtains the FDC XRPPayment proof and calls executeDirectMintingWithData.
7. The personal account approves FXRP and calls PayMorphRouter atomically.
8. PayMorphRouter distributes FXRP directly or swaps to USDT0 and distributes it.
9. Receipt page shows the complete cross-chain evidence.

1.4 Business model

- Protocol/application fee: default 0.50% (50 basis points), maximum contract-enforced fee 3.00% (300 basis points).
- Starter plan: hosted payment links and basic receipts.
- Business plan: branded checkout, API keys, webhooks, reusable payout rules, and reporting.
- Platform plan: marketplace SDK, multiple recipients, affiliates, and per-recipient settlement preferences in a later release.

Hackathon rule

Implement the fee mechanism on testnet but clearly label all test tokens as valueless. Do not charge real users or imply that a testnet transaction is a legal payment.

2. Users, jobs, and requirements

2.1 Roles

Role	Needs	MVP permissions
Merchant owner	Create invoices, choose settlement, see payments	Wallet sign-in; CRUD own invoices; view receipts
Payer	Pay one invoice with native XRP	Xaman identify; sign payment; see status/receipt
Recipient	Receive FXRP or USDT0	No account required; valid Coston2 address
Operator	Maintain executor and investigate failures	Restricted admin dashboard; recovery tools
Public viewer	Verify receipt	Read-only receipt URL and explorer links

2.2 Primary jobs-to-be-done

- As a merchant, I want to request a fixed amount and avoid asking the customer to bridge or acquire FLR.
- As a merchant, I want to choose whether I receive XRP exposure (FXRP) or stable settlement (USDT0).
- As a marketplace, I want one customer payment to pay the seller, affiliate, and platform automatically.
- As a payer, I want to review one clear XRP amount and approve it in Xaman.
- As either party, I want proof of what happened on both chains.
- As an operator, I want deterministic recovery when a direct mint becomes stuck.

2.3 Functional requirements

ID	Requirement	Acceptance summary
FR-001	Merchant wallet authentication	Challenge/response sign-in on Coston2-compatible EVM wallet.
FR-002	Invoice creation	XRP or USD denomination, FXRP or USDT0 settlement, 1-10 recipients.
FR-003	Percentage validation	Recipient basis points must total exactly 10,000.
FR-004	Public checkout	Human-readable invoice, merchant identity, expiry, settlement disclosure.
FR-005	Payer identification	Xaman SignIn before quote/payment construction.
FR-006	Quote creation	Read FTSO, protocol fees, nonce, direct-mint fees, pool settings, and expiry.
FR-007	Xaman payment payload	XRPL Payment to current direct-mint address, no DestinationTag, exact Amount, 0xFE memo.
FR-008	Off-chain userOp delivery	Store exact ABI-encoded PackedUserOperation committed by memo hash.
FR-009	XRPL validation	Confirm validated tesSUCCESS transaction and verify every critical field.
FR-010	FDC finalization	Obtain XRPPayment proof and submit executeDirectMintingWithData.
FR-011	Atomic settlement	Approve + settle calls execute atomically from personal account.
FR-012	FXRP settlement	Distribute invoice amount and fee in FXRP.
FR-013	USDT0 settlement	Swap exact output within max FXRP input,

ID	Requirement	Acceptance summary
		distribute USDT0, refund unused FXRP.
FR-014	Receipt	Persist and render XRPL/Flare hashes, amounts, recipients, fees, timestamps.
FR-015	Recovery	Diagnose and prepare official 0xE0 skip-memo recovery flow.
FR-016	Admin operations	Pause app/contract, disable USDT0 route, inspect jobs and retry safe steps.

2.4 Non-functional requirements

- Correctness over speed: never mark paid from a webhook alone.
- Availability target for hackathon demo: web and executor restart safely without losing state.
- Accessibility: keyboard navigation, visible focus, semantic headings, form labels, status not conveyed by color alone.
- Responsive design: checkout works on mobile because payer may switch to Xaman on the same device.
- Security: no private keys in browser or repository; secrets server-side only; executor key least-privileged and testnet-funded.
- Auditability: structured logs contain attempt ID and chain identifiers but never secrets or full user-operation plaintext when unnecessary.
- Determinism: all amount calculations use BigInt/integer base units and documented rounding.

3. Scope and non-goals

3.1 Must-have MVP

- [x] Merchant sign-in and dashboard.
- [x] Create, publish, expire, and cancel invoices.
- [x] XRP and USD invoice denominations.
- [x] FXRP settlement.
- [x] USDT0 settlement behind a runtime capability check.
- [x] One to ten recipient split.
- [x] Xaman SignIn and Payment payloads.
- [x] 0xFE custom instruction and executor.
- [x] XRPL Testnet and Coston2 explorer links.
- [x] Receipt and payment history.
- [x] Operator recovery page.
- [x] Unit, integration, contract, and end-to-end tests.
- [x] Hosted deployment and public GitHub repository.

3.2 Explicit non-goals for hackathon

- Fiat bank-account payouts or licensed off-ramping.
- KYC/AML decisioning.
- Mainnet real-value launch.
- Recurring subscriptions.
- Refunds after a successful merchant settlement.
- Chargebacks or buyer disputes.
- Multiple source assets beyond XRP.
- Multiple DEX aggregators.
- Custody of user private keys.

- Native mobile app.
- Full accounting, tax, or invoice-compliance suite.

3.3 Stretch features only after the core works

- Reusable payout templates.
- Merchant API keys and webhooks.
- Embeddable checkout SDK.
- Email/Telegram notifications.
- Recipient choice of settlement asset.
- Public product catalog demo.
- Xaman xApp packaging.
- Mainnet deployment configuration.

4. End-to-end product flows

4.1 Merchant onboarding

1. Merchant opens /login and connects a Coston2 EVM wallet.
2. Frontend requests a one-time nonce from POST /api/auth/nonce.
3. Merchant signs an EIP-191 message containing domain, wallet, nonce, issuedAt, expiration, and chain ID 114.
4. Backend verifies signature and creates an HttpOnly, Secure, SameSite=Lax session cookie.
5. Merchant completes display name, optional logo, and default settlement preferences.
6. Dashboard shows Coston2 network badge and faucet links for test assets.

4.2 Merchant creates invoice

1. Enter title, description, external reference, denomination (USD/XRP), amount, expiry, and settlement asset.
2. Add recipients. Each has label, Coston2 address, and basis points. Default single recipient = merchant wallet at 10,000 bps.
3. UI validates addresses, positive amount, expiry, and exact 10,000-bps sum.
4. Backend stores draft; merchant publishes it; public slug becomes active.
5. No quote is stored at invoice creation because rates and protocol parameters can change.

4.3 Customer checkout - mandatory two-step Xaman flow

Why identification comes first

The 0xFE user operation commits to a sender personal account and nonce derived from the actual XRPL account. Xaman normally fills the Account field at signing, so PayMorph must first identify the payer through SignIn (or run as an xApp with known account context) before constructing the payment memo.

1. Customer opens /pay/{slug}.
2. Customer clicks Connect Xaman.
3. Backend creates a Xaman SignIn payload; UI displays QR/deeplink.
4. After signing, backend fetches payload details, verifies the signature/account, and creates a short-lived payer session.
5. Customer clicks Get quote.
6. Backend derives personal account, reads nonce, FTSO price, FAssets fees/settings, direct-mint address, executor fee, and settlement route.
7. Backend encodes Call[] and PackedUserOperation, stores the bytes, computes the 0xFE memo, and returns the XRP amount and expiry.

8. Customer clicks Pay with XRP; backend creates the Xaman Payment payload tied to the identified account/user token.
9. Customer signs; app transitions to cross-chain status page.

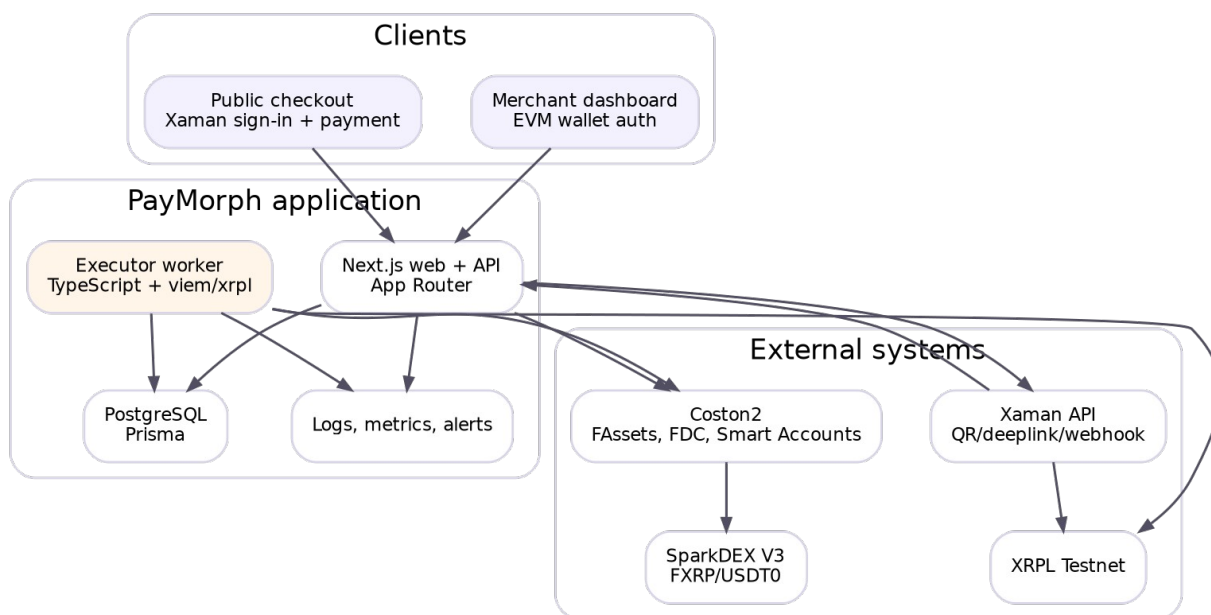
4.4 Cross-chain processing

1. Xaman webhook is treated as a notification only. Backend fetches authoritative payload details.
2. Worker waits for the XRPL transaction to be validated and tesSUCCESS.
3. Worker verifies destination, amount, memo, absence of DestinationTag, network, payer account, and expiry.
4. Worker requests/retrieves the FDC XRPPayment proof using the official Flare starter utilities.
5. Worker calls AssetManagerFXRP.executeDirectMintingWithData(proof, userOpBytes).
6. AssetManager mints FXRP to the personal account; MasterAccountController verifies the memo commitment and executes the calls.
7. PayMorphRouter settles atomically and emits events.
8. Indexer/worker reads events, updates database, and produces receipt.

4.5 Recipient distribution examples

Scenario	Input	Output
Single merchant, FXRP	Customer sends enough XRP for 100 FXRP gross	Fee to treasury; remainder FXRP to merchant
Single merchant, USDT0	Customer sends max FXRP input for 100 USDT0 invoice + fee	Exact USDT0 output distributed; unused FXRP refunded
Marketplace split	Seller 8500 bps, affiliate 1000 bps, platform 500 bps	Invoice amount distributed by bps; PayMorph service fee handled separately

5. System architecture



5.1 Components

Component	Technology	Responsibility
Web application	Next.js App Router, TypeScript, React, Tailwind	Marketing, merchant dashboard, checkout, API routes, server rendering.
Merchant wallet layer	wagmi + viem	EVM wallet connection, network checks, signed authentication.
Xaman adapter	Official Xaman JS/TS SDK	SignIn, Payment payload, QR/deeplink, webhook verification.
XRPL client	xrpl.js	Validated transaction reads, ledger status, account/address checks.
Flare client	viem + Flare periphery ABIs	Registry lookups, FTSO reads, Smart Account encoding, transaction/event reads.
Database	PostgreSQL + Prisma	Invoices, payer sessions, attempts, jobs, events, audit records.
Executor worker	Node.js TypeScript process	XRPL validation, FDC proof lifecycle, Flare finalization, retries, indexing.
Contracts	Solidity + Foundry	Settlement, splits, fee, DEX adapter, replay protection, pause.
Observability	Pino/OpenTelemetry-compatible logs + error tracker	Correlated logs, alerts, metrics, trace identifiers.

5.2 Trust boundaries

- Xaman signs; PayMorph never receives the payer private key.
- Merchant EVM wallet signs authentication only; no customer funds are custodied by the web server.
- The executor can submit a committed user operation but cannot substitute different bytes because the XRPL memo contains keccak256(data).
- PayMorph database is not the source of truth for payment completion; chain events are.
- The DEX route is allowlisted by contract. The merchant cannot provide an arbitrary router address.
- Admin can pause new settlements but cannot replay a payment ID.

5.3 Deployment topology

```

Internet
|
+-- paymorph.example (Next.js web/API)
|   +-- PostgreSQL
|   +-- Xaman API
|   +-- Coston2 RPC (read only)
|
+-- executor.paymorph.internal (worker, no public UI)
    +-- PostgreSQL job queue
    +-- XRPL Testnet WebSocket
    +-- FDC verifier / DA layer
    +-- Coston2 RPC (signed transactions)

On-chain:
XRPL Testnet -> FAssets Core Vault
Coston2 -> AssetManagerFXRP -> MasterAccountController -> Personal Account
        -> PayMorphRouter -> SparkDexV3Adapter -> recipients
  
```

6. Exact cross-chain protocol

6.1 Network constants

Item	Coston2/test value	Rule
Flare RPC	https://coston2-api.flare.network/ext/C/rpc	Environment configurable; health-check at startup.
Flare chain ID	114	Reject wrong merchant wallet network.
Flare explorer	https://coston2-explorer.flare.network	Generate links from config.
XRPL WebSocket	wss://s.altnet.ripple.test.net:51233	Use primary + optional fallback.
Contract Registry	0xaD67FE6660Fb8dFE9d6b1b4240d8650e30F6019	Use registry to resolve AssetManagerFXRP and FtsoV2.
MasterAccountController	Resolve via registry/reference package	Do not rely on a copied address if registry support exists.
USDT0	0xC1A5B41512496B80903D1f32d6dEa3a73212E71F	Config; validate bytecode at startup.
SparkDEX router	0x8a1E35F5c98C4E85B36B7B253222eE1773b2781	Config; validate bytecode.
SparkDEX fee tier	500	0.05%; config and pool-existence check.
XRP/USD feed ID	0x015852502f55534400000000000000000000000000000000	Read decimals and timestamp at runtime.

Source anchors: [S2], [S8], [S9], [S11]

6.2 Runtime discovery sequence

1. Read AssetManagerFXRP from FlareContractRegistry.
2. Read FXRP token address from AssetManagerFXRP.fAsset().
3. Read current AssetManager settings and direct-minting payment address through official ABI/helpers.
4. Read MasterAccountController through registry/reference package.
5. Read FtsoV2 through registry.
6. Validate configured USDT0 and SparkDEX router have deployed bytecode.
7. Check factory pool for FXRP/USDT0 at configured fee tier; cache capability for a short period.
8. Expose resolved values at GET /api/meta/network for diagnostics.

6.3 Personal account and user operation

For the identified XRPL account, derive the deterministic Flare personal account using MasterAccountController.getPersonalAccount. Read its current nonce using getNonce. Construct the personal-account executeUserOp call data around an ordered Call[] batch.

```
Call[] for FXRP settlement
1. FXRP.approve(PayMorphRouter, grossFxrAmount)
2. PayMorphRouter.settleFxr(paymentId, grossFxrAmount,
    recipients, feeBps, deadline, refundTo)

Call[] for USDT0 settlement
1. FXRP.approve(PayMorphRouter, maxFxrInput)
2. PayMorphRouter.settleUsdt0ExactOut(paymentId, maxFxrInput,
    invoiceUsdt0Out, recipients,
    feeBps, poolFee, deadline, refundTo)
```

Each call executes with the personal account as msg.sender. If any call reverts, the entire user operation reverts; partial distribution is impossible. All call.value fields are zero in PayMorph v1, so the executor sends msg.value = 0 to executeDirectMintingWithData.

Source anchors: [S4], [S5]

6.4 0xFE memo construction

Bytes	Field	PayMorph value
0	instructionId	0xFE
1	walletId	Configured wallet ID; 0 until registered
2-9	executorFeeUBA	uint64 executor fee in FXRP base units
10-41	userOpHash	keccak256(abi.encode(PackedUserOperation))

Store both the 42-byte memo and the exact ABI-encoded user operation. The executor receives the XRPL transaction hash, userOp bytes, and totalCallValue (zero) through an authenticated internal job record. Never reconstruct the bytes after the payer signs; a single byte change breaks the commitment.

6.5 XRP amount sent by payer

```

xrplPaymentUBA = netMintUBA + protocolMintFeeUBA + executorFeeUBA
xrplAmountDrops = xrplPaymentUBA // XRP and FXRP use 6-decimal base units

For USDT0 exact-output settlement:
netMintUBA = maxFxrInput

For FXRP settlement:
netMintUBA = grossFxrAmount

```

Read protocol fees from current AssetManager settings/helpers. Never use a static percentage copied into the application. The quote stores a snapshot of every component and becomes invalid after its expiry.

6.6 XRPL Payment template

```

{
  "TransactionType": "Payment",
  "Destination": "<current FAssets direct-minting Core Vault address>",
  "Amount": "<exact drops as decimal string>",
  "Memos": [{
    "Memo": {
      "MemoData": "<0xFE memo hex without 0x for XRPL JSON>"
    }
  ]},
  "LastLedgerSequence": "<short bounded lifetime>"
}

Rules:
- Do not include DestinationTag.
- Account, Fee, and Sequence may be filled by Xaman.
- Force XRPL Testnet in the Xaman payload.
- Tie payload custom metadata to paymentAttemptId and quoteId.

```

6.7 Finalization

1. Executor confirms the exact signed transaction and validates it on XRPL.
2. Executor obtains an XRPPayment attestation proof using official Flare FDC utilities.
3. Executor calls AssetManagerFXRP.executeDirectMintingWithData(proof, userOpBytes).
4. On-chain code checks XRPL ownership/proof binding, memo hash, sender, and nonce.
5. FXRP mint and user-operation settlement happen in the same Flare transaction.
6. Worker requires DirectMintingExecuted/UserOperationExecuted plus PayMorph PaymentSettled evidence before success.

Source anchors: [S4], [S5], [S6]

7. Smart-contract specification

7.1 Contract set

Contract	Purpose
PayMorphRouter	Entry point for FXRP and USDT0 settlement; replay protection, fee, split, refund, events, pause.
SparkDexV3Adapter	Allowlisted DEX adapter that swaps FXRP for exact USDT0 output and returns unused FXRP.
IPayMorphSettlementAdapter	Adapter interface for future DEX or market-maker routes.
Deployment scripts	Resolve FXRP, deploy contracts, grant roles, write deployment manifest, verify bytecode.

7.2 Solidity conventions

- Solidity 0.8.25 or repository-pinned compatible version.
- Foundry for build, tests, scripts, coverage, invariant fuzzing.
- OpenZeppelin SafeERC20, ReentrancyGuard, Pausable, AccessControl.
- Non-upgradeable v1 contracts. Deploy a new version rather than adding proxy complexity to the hackathon MVP.
- Custom errors instead of long revert strings.
- NatSpec on every external/public function, event, struct, and security-sensitive invariant.
- Checks-effects-interactions and exact approval reset where token behavior requires it.

7.3 PayMorphRouter data structures

```
enum SettlementAsset { FXRP, USDT0 }

struct Recipient {
    address account;
    uint16 bps;      // all recipients together must equal 10_000
}

mapping(bytes32 => bool) public settled;
uint16 public serviceFeeBps;          // default 50
uint16 public constant MAX_FEE_BPS = 300;
address public feeRecipient;
IERC20 public immutable FXRP;
IERC20 public immutable USDT0;
IPayMorphSettlementAdapter public adapter;
```

7.4 External functions

Function	Key inputs	Required behavior
settleFxr	paymentId, grossFxrAmount, recipients, feeBpsSnapshot, deadline, refundTo	Pull gross FXRP from msg.sender; service fee; split net invoice amount; mark settled; emit events.
settleUsdt0ExactOut	paymentId, maxFxrInput, invoiceUsdt0Out, recipients, feeBpsSnapshot, poolFee, deadline, refundTo	Pull max FXRP; calculate USDT0 fee; swap exact total output; split invoice output; pay fee; refund unused FXRP.
previewServiceFee	amount, feeBps	Pure rounded fee calculation used by tests/UI parity.
setServiceFeeBps	newFee	FEE_MANAGER_ROLE; <= MAX_FEE_BPS; emit change.
setFeeRecipient	address	FEE_MANAGER_ROLE; nonzero; emit change.
setAdapter	address	ADAPTER_MANAGER_ROLE; nonzero/bytecode;

Function	Key inputs	Required behavior
		emit change.
pause/unpause	none	PAUSER_ROLE; blocks settlement.

7.5 Fee semantics

The invoice amount is the amount owed to invoice recipients before the PayMorph service fee. The payer funds both invoice amount and service fee. This prevents the merchant from receiving less than the displayed invoice amount.

```

serviceFee = ceil(invoiceAmount * serviceFeeBps / 10_000)

FXRP mode:
grossFxrAmount = invoiceFxrAmount + serviceFeeFxr
distribute invoiceFxrAmount by recipient bps
transfer serviceFeeFxr to feeRecipient

USDT0 mode:
totalUsdt0Out = invoiceUsdt0Out + serviceFeeUsdt0
adapter exact-output swaps up to maxFxrInput
distribute invoiceUsdt0Out by recipient bps
transfer serviceFeeUsdt0 to feeRecipient
refund maxFxrInput - actualFxrUsed to refundTo

```

7.6 Rounding rules

- Service fee rounds up so the fee is never under-collected by one base unit.
- Recipient payouts are floor(amount * bps / 10,000) for all recipients except the final recipient.
- The final recipient receives the remainder so total payouts equal the invoice amount exactly.
- No dust remains in PayMorphRouter after a successful settlement.
- USDT0 uses its on-chain decimals read/validated at deployment; FXRP base-unit handling must be tested against the deployed token.

7.7 Validation and replay protection

- paymentId != bytes32(0) and settled[paymentId] == false.
- block.timestamp <= deadline.
- 1 <= recipients.length <= 10.
- Every recipient nonzero and unique; bps > 0; total bps == 10,000.
- Amount > 0; maxFxrInput > 0; invoiceUsdt0Out > 0.
- feeBpsSnapshot equals current serviceFeeBps, or contract explicitly accepts a lower signed snapshot.
Recommended v1: exact equality.
- Configured adapter and token addresses are nonzero.
- USDT0 route only with allowlisted fee tier.
- nonReentrant and whenNotPaused.

7.8 Events

```

event PaymentSettled(
    bytes32 indexed paymentId,
    address indexed payerPersonalAccount,
    SettlementAsset asset,
    uint256 invoiceAmount,
    uint256 serviceFee,
    uint256 inputFxrUsed,
    address refundTo,
    uint256 refundFxr
);

event RecipientPaid(

```

```

bytes32 indexed paymentId,
address indexed recipient,
address indexed token,
uint256 amount,
uint16 bps
);

event AdapterUpdated(address indexed oldAdapter, address indexed newAdapter);
event ServiceFeeUpdated(uint16 oldFeeBps, uint16 newFeeBps);
event FeeRecipientUpdated(address indexed oldRecipient, address indexed newRecipient);

```

7.9 Adapter interface

```

interface IPayMorphSettlementAdapter {
    function swapExactOutput(
        address tokenIn,
        address tokenOut,
        uint256 amountOut,
        uint256 amountInMaximum,
        uint24 poolFee,
        uint256 deadline
    ) external returns (uint256 amountIn);
}

```

SparkDexV3Adapter is only callable by PayMorphRouter. It receives max input from the router, approves the official router, calls exactOutputSingle, returns exact USDT0 output and unused FXRP to PayMorphRouter, resets approval, and returns actual amountIn. It rejects unsupported token pairs and fee tiers.

7.10 Contract acceptance tests

- FXRP single-recipient settlement exact balances.
- FXRP 10-recipient settlement exact totals and final-recipient remainder.
- USDT0 exact-output swap and unused FXRP refund.
- Duplicate paymentId reverts.
- Expired deadline reverts.
- Recipient sum 9,999 or 10,001 reverts.
- Zero/duplicate recipient reverts.
- Fee > maximum reverts.
- Paused settlement reverts.
- Malicious adapter reentrancy fails.
- Fee-on-transfer/unexpected token behavior is rejected or documented unsupported.
- Fuzz invariant: router core-token balance returns to zero after successful settlement.
- Fuzz invariant: total recipient payout + service fee equals exact output.
- Atomic failure: adapter revert leaves no recipient partially paid.

8. Application services

8.1 Web/API service modules

Module	Responsibilities
auth	Merchant nonce/signature verification; session cookie; authorization guards.
merchant	Profile, defaults, branding, webhook configuration.
invoice	CRUD, slug generation, recipient validation, publish/cancel/expire.
payer-session	Xaman SignIn lifecycle and XRPL account binding.

Module	Responsibilities
quote	FTSO reads, route capability, amounts, fees, personal account, nonce, userOp encoding.
xaman	Payload creation/get, webhook confirmation, QR/deeplink data.
network	Flare registry discovery, XRPL clients, health checks, address cache.
receipt	Chain event normalization and immutable public receipt view.
admin	Job inspection, route toggle, pause signals, recovery preparation.

8.2 Executor worker modules

Module	Responsibilities
job-runner	Claim due jobs with database lock; heartbeat; retries; dead-letter state.
xrpl-validator	Fetch validated transaction and compare exact expected fields.
fdc-client	Prepare/request/retrieve XRPPayment proof using official Flare utilities.
flare-finalizer	Simulate and submit executeDirectMintingWithData; wait confirmations.
event-indexer	Decode AssetManager, Smart Account, PayMorph events; update attempt.
recovery-diagnostics	Check isTransactionIdUsed and determine official 0xE0 eligibility.
reconciler	Periodically compare database pending attempts against chain state.

8.3 Job execution policy

- One active job per payment attempt and step.
- Claim with SELECT ... FOR UPDATE SKIP LOCKED or Prisma equivalent transaction.
- Store attempts, nextRunAt, lastErrorCode, lastErrorMessage, and lockedUntil.
- Exponential backoff with jitter for transient RPC/API failures.
- No automatic retry for deterministic validation mismatch, invalid memo, wrong amount, or contract revert caused by expired quote.
- Every signed Flare submission records nonce, raw transaction hash, and replacement history.
- Worker restarts must resume safely from persisted states.

9. Database schema

9.1 Entity overview

Entity	Purpose	Important uniqueness
Merchant	Owner profile and wallet	walletAddress unique
Session	Merchant login session	tokenHash unique
Invoice	Requested payment	publicSlug unique
InvoiceRecipient	Split rule	invoiceId + position
PayerSession	Xaman-identified XRPL user	sessionTokenHash unique

Entity	Purpose	Important uniqueness
Quote	Immutable amount/protocol snapshot	id; one selected per attempt
PaymentAttempt	Cross-chain lifecycle	paymentId unique; xrplTxHash unique nullable
XamanPayload	SignIn/Payment payload metadata	payloadUuid unique
ExecutorJob	Durable work queue	attemptId + jobType + generation
ChainEvent	Normalized event log	chainId + txHash + logIndex unique
WebhookEvent	Raw webhook receipt/idempotency	provider + deliveryKey unique
AuditLog	Operator/merchant security trail	append-only ID
Deployment	Contract/network manifest	network + version unique

9.2 Prisma schema (normative starting point)

```

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

enum InvoiceStatus { DRAFT ACTIVE EXPIRED CANCELLED }
enum Denomination { XRP USD }
enum SettlementAsset { FXRP USDT0 }
enum PayloadKind { SIGN_IN PAYMENT RECOVERY }
enum PayloadStatus { CREATED OPENED SIGNED REJECTED EXPIRED }
enum AttemptStatus {
  CREATED IDENTIFYING IDENTIFIED QUOTED XAMAN CREATED AWAITING_SIGNATURE
  XRPL_SIGNED XRPL_VALIDATED USEROP_UPLOADED FDC_REQUESTED FDC_READY
  FLARE_SUBMITTED FLARE_CONFIRMED SETTLED REJECTED QUOTE_EXPIRED
  XRPL_FAILED EXECUTION_REVERTED RECOVERY_REQUIRED RECOVERED CANCELLED
}
enum JobType { VALIDATE_XRPL REQUEST_FDC SUBMIT_FLARE INDEX_EVENTS RECONCILE RECOVERY_DIAGNOSIS }
enum JobStatus { READY RUNNING RETRY SUCCEEDED DEAD }

model Merchant {
  id          String   @id @default(uuid())
  walletAddress String   @unique
  displayName String
  logoUrl     String?
  defaultAsset SettlementAsset @default(FXRP)
  webhookUrl  String?
  webhookSecretEnc String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  invoices    Invoice[]
  sessions    Session[]
  auditLogs   AuditLog[]
}

model Session {
  id          String   @id @default(uuid())
  merchantId  String
  tokenHash   String   @unique
  expiresAt   DateTime
  createdAt   DateTime @default(now())
  merchant    Merchant @relation(fields:[merchantId], references:[id], onDelete: Cascade)
  @@index([merchantId, expiresAt])
}

model AuthNonce {
  id          String   @id @default(uuid())
  walletAddress String
  nonceHash   String
  expiresAt   DateTime
  usedAt      DateTime?
  createdAt   DateTime @default(now())

```

```

    @@index([walletAddress, expiresAt])
}

model Invoice {
  id          String    @id @default(uuid())
  merchantId  String
  publicSlug  String    @unique
  title       String
  description  String?
  externalRef String?
  denomination Denomination
  amountBaseUnits Decimal @db.Decimal(78,0)
  settlementAsset SettlementAsset
  status       InvoiceStatus @default(DRAFT)
  expiresAt    DateTime
  publishedAt  DateTime?
  cancelledAt  DateTime?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
  merchant     Merchant @relation(fields:[merchantId], references:[id])
  recipients   InvoiceRecipient[]
  attempts     PaymentAttempt[]
  @@index([merchantId, createdAt])
  @@index([status, expiresAt])
}

model InvoiceRecipient {
  id          String    @id @default(uuid())
  invoiceId   String
  position    Int
  label       String
  address     String
  bps         Int
  invoice     Invoice @relation(fields:[invoiceId], references:[id], onDelete: Cascade)
  @@unique([invoiceId, position])
  @@index([invoiceId])
}

model PayerSession {
  id          String    @id @default(uuid())
  sessionTokenHash String @unique
  xrplAccount String?
  xamanUserTokenEnc String?
  signInPayloadId String?
  expiresAt   DateTime
  createdAt   DateTime @default(now())
  attempts    PaymentAttempt[]
}

model Quote {
  id          String    @id @default(uuid())
  invoiceId   String
  payerXrplAccount String
  personalAccount String
  personalAccountNonce Decimal @db.Decimal(78,0)
  xrpUsdValue Decimal? @db.Decimal(78,0)
  xrpUsdDecimals Int?
  xrpUsdTimestamp DateTime?
  invoiceOutBaseUnits Decimal @db.Decimal(78,0)
  serviceFeeOutBaseUnits Decimal @db.Decimal(78,0)
  maxFxrplInputUBA Decimal @db.Decimal(78,0)
  protocolMintFeeUBA Decimal @db.Decimal(78,0)
  executorFeeUBA Decimal @db.Decimal(78,0)
  xrplPaymentDrops Decimal @db.Decimal(78,0)
  slippageBps Int
  route       String
  poolFee     Int?
  userOpHash  String
  userOpDataEnc String
  memoHex     String
  directMintAddress String
  expiresAt   DateTime
  createdAt   DateTime @default(now())
  invoice     Invoice @relation(fields:[invoiceId], references:[id])
  attempt     PaymentAttempt?
  @@index([invoiceId, createdAt])
}

```

```

model PaymentAttempt {
  id                String    @id @default(uuid())
  paymentId         String    @unique
  invoiceId         String
  payerSessionId    String
  quoteId           String    @unique
  status            AttemptStatus @default(CREATED)
  payerXrplAccount  String
  personalAccount   String
  xamanPayloadUuid  String?   @unique
  xrplTxHash        String?   @unique
  xrplLedgerIndex   BigInt?
  flareTxHash       String?   @unique
  failureCode       String?
  failureMessage    String?
  recoveryTxHash    String?
  settledAt         DateTime?
  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt
  invoice           Invoice    @relation(fields:[invoiceId], references:[id])
  payerSession      PayerSession @relation(fields:[payerSessionId], references:[id])
  quote            Quote      @relation(fields:[quoteId], references:[id])
  payloads          XamanPayload[]
  jobs              ExecutorJob[]
  chainEvents       ChainEvent[]
  @@index([invoiceId, createdAt])
  @@index([status, updatedAt])
}

model XamanPayload {
  id                String    @id @default(uuid())
  attemptId         String?
  payerSessionId    String?
  kind              PayloadKind
  payloadUuid       String    @unique
  status            PayloadStatus @default(CREATED)
  qrPngUrl          String?
  deeplinkUrl       String?
  txId              String?
  expiresAt         DateTime
  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt
  attempt           PaymentAttempt? @relation(fields:[attemptId], references:[id])
  @@index([attemptId, kind])
}

model ExecutorJob {
  id                String    @id @default(uuid())
  attemptId         String
  jobType           JobType
  generation        Int       @default(0)
  status            JobStatus @default(READY)
  attempts          Int       @default(0)
  maxAttempts       Int       @default(12)
  nextRunAt         DateTime @default(now())
  lockedBy          String?
  lockedUntil       DateTime?
  lastErrorCode     String?
  lastError         String?
  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt
  attempt           PaymentAttempt @relation(fields:[attemptId], references:[id])
  @@unique([attemptId, jobType, generation])
  @@index([status, nextRunAt])
}

model ChainEvent {
  id                String    @id @default(uuid())
  attemptId         String
  chain             String
  chainId           String
  txHash            String
  logIndex          Int
  blockNumber       BigInt
  eventName         String
  payloadJson       Json
  createdAt         DateTime @default(now())
  attempt           PaymentAttempt @relation(fields:[attemptId], references:[id])
}

```

```

    @@unique([chainId, txHash, logIndex])
    @@index([attemptId, eventName])
  }

  model WebhookEvent {
    id          String    @id @default(uuid())
    provider    String
    deliveryKey String
    headersJson Json
    bodyJson    Json
    processedAt DateTime?
    createdAt   DateTime @default(now())
    @@unique([provider, deliveryKey])
  }

  model AuditLog {
    id          String    @id @default(uuid())
    merchantId  String?
    actorType   String
    actorId     String?
    action      String
    entityType  String
    entityId    String?
    ipHash      String?
    metadata    Json?
    createdAt   DateTime @default(now())
    merchant    Merchant? @relation(fields:[merchantId], references:[id])
    @@index([merchantId, createdAt])
  }

  model Deployment {
    id          String    @id @default(uuid())
    network     String
    version     String
    routerAddress String
    adapterAddress String
    fxrpAddress  String
    usdt0Address String
    swapRouter   String
    deployTxHash String
    manifestJson Json
    createdAt    DateTime @default(now())
    @@unique([network, version])
  }

```

9.3 Data rules

- Store blockchain integers as Decimal(78,0) or strings; convert to BigInt at service boundaries.
- Normalize EVM addresses to checksum form for display and lowercase for indexed comparison if a separate normalized column is needed.
- Store XRPL r-address exactly as validated; do not lowercase it.
- Encrypt Xaman user tokens, webhook secrets, and userOp bytes at rest using AES-256-GCM with key rotation metadata or a managed KMS.
- Never store merchant or payer private keys.
- WebhookEvent is append-only and can be pruned after a retention period; settlement records are retained.
- A Quote is immutable. Create a new quote rather than updating amounts or nonce.

10. HTTP API specification

All JSON responses use { data, error, requestId }. Errors have stable machine codes. Merchant routes require the HttpOnly session cookie; payer routes use a short-lived payer-session cookie. Mutation routes accept Idempotency-Key where noted.

10.1 Common envelope

```
// success
{ "data": { ... }, "error": null, "requestId": "req_..." }

// error
{
  "data": null,
  "error": {
    "code": "QUOTE_EXPIRED",
    "message": "The quote has expired. Request a new quote.",
    "details": { "expiredAt": "..." }
  },
  "requestId": "req_..."
}
```

10.2 Route inventory

Method	Route	Auth	Purpose
POST	/api/auth/nonce	Public	Create merchant wallet-auth challenge.
POST	/api/auth/verify	Public	Verify EVM signature and create merchant session.
POST	/api/auth/logout	Merchant	Revoke current session.
GET	/api/me	Merchant	Merchant profile and network capability.
PATCH	/api/me	Merchant	Update display name/logo/defaults.
GET	/api/invoices	Merchant	List own invoices with filters/cursor.
POST	/api/invoices	Merchant + idempotency	Create draft invoice.
GET	/api/invoices/{id}	Merchant	Invoice detail and attempts.
PATCH	/api/invoices/{id}	Merchant	Edit draft or allowed fields.
POST	/api/invoices/{id}/publish	Merchant + idempotency	Activate invoice.
POST	/api/invoices/{id}/cancel	Merchant + idempotency	Cancel unpaid invoice.
GET	/api/public/invoices/{slug}	Public	Checkout-safe invoice data.
POST	/api/payer/signin	Public + idempotency	Create Xaman SignIn payload.
GET	/api/payer/signin/{payloadId}	Payer	Return authoritative SignIn status.
POST	/api/public/invoices/{slug}/quotes	Payer + idempotency	Create immutable quote and userOp.
POST	/api/quotes/{quoteId}/payment-payload	Payer + idempotency	Create Xaman Payment payload and attempt.
GET	/api/attempts/{id}	Payer or merchant	Current normalized status.
GET	/api/attempts/{id}/events	Payer or merchant	Server-Sent Events status stream.
GET	/api/receipts/{id}	Public	Public receipt projection.
POST	/api/webhooks/xaman	Provider	Idempotent webhook ingestion.
GET	/api/meta/network	Public	Resolved addresses and capabilities, no secrets.
GET	/api/health	Public	Process liveness.
GET	/api/ready	Infrastructure	DB/RPC/config readiness.
GET	/api/admin/attempts	Operator	Failure/pending search.
POST	/api/admin/attempts/{id}/retry	Operator + idempotency	Retry allowed next step.
POST	/api/admin/attempts/{id}/diagnose-	Operator	Check 0xE0 eligibility.

Method	Route	Auth	Purpose
	recovery		
POST	/api/attempts/{id}/recovery-payload	Payer + idempotency	Create official recovery Xaman payload.

10.3 Create invoice

```

POST /api/invoices
Idempotency-Key: <uuid>

{
  "title": "UI design kit",
  "description": "Commercial license",
  "externalRef": "ORDER-1042",
  "denomination": "USD",
  "amount": "25.00",
  "settlementAsset": "USDT0",
  "expiresAt": "2026-08-10T18:30:00.000Z",
  "recipients": [
    { "label": "Creator", "address": "0x...", "bps": 8500 },
    { "label": "Affiliate", "address": "0x...", "bps": 1000 },
    { "label": "Platform", "address": "0x...", "bps": 500 }
  ]
}

201 response:
{
  "data": {
    "id": "...", "publicSlug": "ui-design-kit-a8f4",
    "status": "DRAFT", "amountBaseUnits": "2500"
  }, "error": null, "requestId": "req_..."
}

```

10.4 Xaman SignIn

```

POST /api/payer/signin
Idempotency-Key: <uuid>

{ "invoiceSlug": "ui-design-kit-a8f4" }

201 response:
{
  "data": {
    "payerSessionId": "...",
    "payloadUuid": "...",
    "qrPngUrl": "https://...",
    "deeplinkUrl": "https://...",
    "expiresAt": "..."
  }, "error": null, "requestId": "req_..."
}

After signature, GET status returns:
{
  "data": {
    "status": "SIGNED",
    "xrplAccount": "r...",
    "network": "TESTNET"
  }, "error": null, "requestId": "req_..."
}

```

10.5 Quote creation

```

POST /api/public/invoices/{slug}/quotes
Idempotency-Key: <uuid>

```

```
{ "slippageBps": 150 }

201 response:
{
  "data": {
    "quoteId": "...",
    "invoiceAmount": { "asset": "USDT0", "baseUnits": "25000000", "display": "25.00" },
    "serviceFee": { "asset": "USDT0", "baseUnits": "125000", "display": "0.125" },
    "customerPays": { "asset": "XRP", "drops": "...", "display": "..." },
    "maxFxrInputUBA": "...",
    "route": "SPARKDEX_V3_EXACT_OUTPUT",
    "personalAccount": "0x...",
    "userOpHash": "0x...",
    "expiresAt": "...",
    "warnings": []
  }, "error": null, "requestId": "req_..."
}
```

10.6 Payment payload

```
POST /api/quotes/{quoteId}/payment-payload
Idempotency-Key: <uuid>

{}

201 response:
{
  "data": {
    "attemptId": "...",
    "paymentId": "0x...",
    "payloadUuid": "...",
    "qrPngUrl": "https://...",
    "deeplinkUrl": "https://...",
    "statusUrl": "/pay/.../status/...",
    "expiresAt": "..."
  }, "error": null, "requestId": "req_..."
}
```

10.7 Attempt status

```
GET /api/attempts/{id}

{
  "data": {
    "id": "...",
    "status": "FDC_READY",
    "progress": [
      { "step": "XAMAN_SIGNATURE", "state": "DONE", "at": "..." },
      { "step": "XRPL_VALIDATION", "state": "DONE", "txHash": "..." },
      { "step": "FDC_PROOF", "state": "DONE" },
      { "step": "FLARE_SETTLEMENT", "state": "RUNNING" }
    ],
    "recoverable": false,
    "nextPollAfterMs": 4000
  }, "error": null, "requestId": "req_..."
}
```

10.8 Error codes

Code	Meaning
VALIDATION_ERROR	Malformed request or recipient sum/address failure.
UNAUTHENTICATED	Missing/expired merchant or payer session.
FORBIDDEN	Resource owned by another merchant/operator only.

Code	Meaning
INVOICE_NOT_ACTIVE	Draft, expired, cancelled, or already constrained.
PAYER_NOT_IDENTIFIED	Quote requested before Xaman SignIn.
WRONG_XRPL_NETWORK	Xaman account/signature is not on XRPL Testnet.
QUOTE_ROUTE_UNAVAILABLE	USDT0 pool or route disabled/insufficient.
QUOTE_EXPIRED	Quote or nonce no longer usable.
NONCE_CHANGED	Personal-account nonce changed before payment payload.
XAMAN_REJECTED	User rejected sign request.
XRPL_VALIDATION_MISMATCH	Destination/amount/memo/account differs.
FDC_NOT_READY	Proof not finalized yet; retryable.
FLARE_SIMULATION_FAILED	Finalization simulation predicts revert.
SETTLEMENT_REVERTED	Flare settlement reverted; inspect recovery.
RECOVERY_NOT_ELIGIBLE	Transaction already used or conditions not met.
IDEMPOTENCY_CONFLICT	Same key used with a different request body.
RATE_LIMITED	Client or provider rate limit.
INTERNAL_ERROR	Unexpected failure; requestId for support.

10.9 API security controls

- CSRF protection for cookie-authenticated merchant mutations.
- Strict Origin/Host validation.
- Rate limits by IP hash, merchant ID, payer session, and route class.
- Idempotency key stored with canonical request-body hash and response.
- Xaman webhook body size limit and signature/provider confirmation.
- No userOp bytes returned to public clients after creation.
- Admin routes require separate operator identity and optional IP allowlist.
- Zod schemas shared between API and client; unknown fields rejected for sensitive routes.

11. Web routes and page behavior

Route	Audience	Required behavior
/	Public	Hero, how it works, testnet disclaimer, CTA to merchant login and demo checkout.
/login	Public	Connect EVM wallet; switch to Coston2; sign challenge.
/dashboard	Merchant	KPI cards, recent payments, route health, quick create.
/dashboard/invoices	Merchant	Filterable invoice table; status, amount, settlement, paid attempts.
/dashboard/invoices/new	Merchant	Multi-step invoice form with recipient split builder.
/dashboard/invoices/{id}	Merchant	Invoice detail, share link, attempts, receipts, cancel/publish.
/dashboard/payments	Merchant	Cross-invoice attempt history and export CSV.
/dashboard/settings	Merchant	Profile, defaults, fee display, webhook settings,

Route	Audience	Required behavior
		testnet addresses.
/pay/{slug}	Public/Payer	Invoice, Connect Xaman, quote, payment CTA, QR/deeplink.
/pay/{slug}/status/{attemptId}	Payer	Live cross-chain progress, timeout guidance, recovery link.
/receipt/{attemptId}	Public	Immutable human receipt with both explorer links and split.
/recover/{attemptId}	Payer	Explain stuck mint; eligibility; create and sign 0xE0 recovery payload.
/docs	Public	Product/API overview and testnet guide.
/docs/api	Public	API reference generated from OpenAPI.
/privacy	Public	Data handling disclosure.
/terms	Public	Testnet-only terms and no-value disclaimer.
/admin/ops	Operator	Queue health, pending/stuck attempts, route toggle, recovery diagnosis.

11.1 Checkout page states

State	Primary UI	Actions
Unidentified	Invoice summary + Connect Xaman	Open SignIn QR/deeplink.
Identifying	QR, mobile deeplink, waiting indicator	Cancel or retry after expiry.
Identified	Show r-address shortened and settlement explanation	Get quote.
Quoted	Exact XRP amount, service fee, quote timer, recipient preview	Pay with XRP or refresh quote.
Awaiting signature	Xaman Payment QR/deeplink	Watch payload status.
Processing	Step timeline across XRPL/FDC/Flare	Keep page open or copy status URL.
Settled	Success, amounts, receipt/explorer links	Open receipt.
Failed recoverable	Clear reason and value-safety explanation	Start recovery.
Failed terminal	Validation mismatch/rejection	Create new quote/attempt; contact support with requestId.

11.2 Invoice form fields

Field	Type/rules	Default
Title	1-80 chars	Required
Description	0-500 chars	Optional
External reference	0-80 chars	Optional
Denomination	XRP or USD	USD
Amount	Positive decimal; max by environment policy	Required
Settlement asset	FXRP or USDT0	Merchant default
Expiry	15 minutes to 30 days from now	24 hours

Field	Type/rules	Default
Recipient label	1-50 chars	Merchant
Recipient address	Valid EVM address	Merchant wallet
Recipient bps	1-10,000; total 10,000	10,000
Public note	0-280 chars; no secrets	Optional

11.3 Receipt fields

- PayMorph payment ID and status.
- Merchant/invoice title and external reference.
- Payer XRPL account (full or privacy-masked with reveal option; blockchain is public).
- Original XRP amount and drop value.
- Invoice denomination and amount.
- Settlement asset, service fee, slippage/refund.
- Recipient labels, addresses, basis points, and amounts.
- XRPL transaction hash and explorer URL.
- Flare finalization transaction hash and explorer URL.
- Quote timestamp, payment time, settlement time.
- Contract version and router address.
- Testnet/no-real-value badge.

12. UI system and components

12.1 Design principles

- Explain the user goal, not protocol jargon. “Pay with XRP” is primary; FAssets details live in an expandable panel.
- Always distinguish XRP on XRPL from FXRP on Flare.
- Every amount includes asset symbol and network label.
- Never use “instant” because FDC finalization may take minutes.
- Show a persistent quote expiry timer and a fresh-quote action.
- On mobile, prioritize Xaman deeplink; on desktop, show QR and deep link.
- Use status copy that makes clear funds are processing, not lost.
- Testnet badges must be visible in header, checkout, and receipt.

12.2 Core components

Component	Behavior
AppShell	Header, testnet badge, navigation, account menu.
NetworkBadge	XRPL Testnet / Coston2 explicit labels.
WalletAuthButton	Merchant EVM connect/switch/sign state.
XamanConnectCard	SignIn QR, deeplink, expiration, status.
InvoiceSummaryCard	Merchant, item, amount, expiry, settlement.
RecipientSplitEditor	Rows, address validation, bps meter totaling 100%.
QuoteCard	XRP due, service fee, max input, timer, refresh.
PaymentQrDialog	Xaman Payment QR/deeplink and payload state.
CrossChainTimeline	XRPL signed/validated, FDC, Flare, settled.
AssetAmount	Base-unit-safe display and symbol/network.

Component	Behavior
ExplorerLink	Network-aware external link and hash copy.
ReceiptCard	Printable receipt with testnet watermark.
RouteHealthBanner	USDT0 unavailable fallback guidance.
RecoveryPanel	Eligibility checks and 0xE0 explanation.
ErrorPanel	Stable code, human action, request ID.
Skeleton/EmptyState	Loading and no-data states.

12.3 Accessibility checklist

- **[x]** All form inputs have programmatic labels and error associations.
- **[x]** QR code has a text deeplink alternative.
- **[x]** Timeline uses text states and icons, not color only.
- **[x]** Dialogs trap focus and close with Escape.
- **[x]** Copy buttons announce success in an aria-live region.
- **[x]** Countdown does not update more than once per second and has a non-live expiration label.
- **[x]** Tables have header cells and mobile card fallback.
- **[x]** Minimum target size 44x44 CSS pixels for primary mobile actions.
- **[x]** Contrast meets WCAG AA.

13. State machines and idempotency

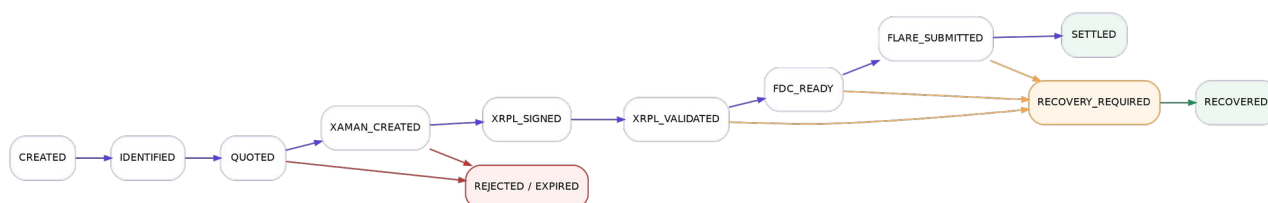


Figure 2. Payment-attempt happy path and recovery branch

13.1 Attempt transitions

From	To	Guard/event
CREATED	IDENTIFYING	SignIn payload created
IDENTIFYING	IDENTIFIED	Authoritative Xaman payload signed and account verified
IDENTIFIED	QUOTED	Immutable quote/userOp created
QUOTED	XAMAN_CREATED	Payment payload created
XAMAN_CREATED	XRPL_SIGNED	Xaman reports signed and txid verified from API
XRPL_SIGNED	XRPL_VALIDATED	XRPL transaction validated tesSUCCESS and exact match
XRPL_VALIDATED	FDC_REQUESTED	FDC request initiated or proof job accepted
FDC_REQUESTED	FDC_READY	Valid proof retrieved
FDC_READY	FLARE_SUBMITTED	Finalization transaction broadcast
FLARE_SUBMITTED	FLARE_CONFIRMED	Transaction confirmed, decoded
FLARE_CONFIRMED	SETTLED	PayMorph PaymentSettled and payouts

From	To	Guard/event
		reconciled

13.2 Invalid transitions

Implement one transition function in the domain layer. It checks an allowlist and performs the database update with an optimistic version field or transaction lock. API handlers and workers must not directly write arbitrary status strings.

13.3 Idempotency strategy

- Canonicalize request body JSON and hash it with route + authenticated actor.
- On first request, reserve key in the same database transaction as resource creation.
- On identical retry, return stored status/body.
- On same key with different body, return IDEMPOTENCY_CONFLICT.
- Xaman webhooks dedupe by payload UUID plus provider delivery key.
- XRPL tx hash is unique across attempts.
- Contract settled[paymentId] prevents on-chain replay even if infrastructure fails.
- Chain events dedupe by chainId, txHash, logIndex.

13.4 Reconciliation loop

- Every 60 seconds, scan nonterminal attempts older than their expected step threshold.
- Re-fetch Xaman payload, XRPL transaction, Flare receipt, and PaymentSettled event as appropriate.
- Advance only forward; never downgrade a terminal status.
- If Flare transaction succeeded but DB missed events, reconstruct receipt from logs.
- If XRPL validated but no finalization after threshold, mark RECOVERY_REQUIRED only after diagnostic checks.

14. Pricing, quotes, fees, and amounts

14.1 Base units

Asset/value	Storage/calculation
XRP	drops, integer; 1 XRP = 1,000,000 drops
FXRP	UBA integer; confirm token decimals from deployment and protocol
USDT0	token base units; read decimals on-chain and assert expected configuration
USD invoice	minor units in MVP, e.g. cents for two-decimal UI; convert to USDT0 base units
Basis points	integer 0-10,000
FTSO values	integer value + returned decimals + timestamp
Timestamps	UTC ISO 8601 in API; Unix seconds on-chain

14.2 Quote TTL

Default quote lifetime: 15 minutes. This is longer than a typical FDC round but still limits price and nonce risk. Make it configurable between 5 and 30 minutes. The payment Xaman payload must expire no later than the quote. Do not create a quote if invoice expiry is earlier.

14.3 USD invoice conversion

```

Definitions (all integer math):
invoiceUsdMinor      merchant-requested USD cents
xrpUsdValue/decimals FTS0 XRP/USD
feeBps               current PayMorph fee
slippageBps          environment/user policy

For USDT0 settlement:
invoiceUsdt0Out = scale USD minor units to USDT0 decimals
serviceFee0Out  = ceil(invoiceUsdt0Out * feeBps / 10_000)
totalUsdt0Out   = invoiceUsdt0Out + serviceFee0Out
dexQuotedFxrIn = exact-output quote or conservative route estimate
maxFxrInInput   = ceil(dexQuotedFxrIn * (10_000 + slippageBps) / 10_000)

For FXRP settlement of a USD invoice:
invoiceFxr = ceil(USD value / XRPUSD price in compatible integer scale)
serviceFeeFxr = ceil(invoiceFxr * feeBps / 10_000)
grossFxr = invoiceFxr + serviceFeeFxr

```

14.4 Price-feed validation

- Read value, decimals, and timestamp in one call.
- Reject nonpositive values.
- Reject feed older than configured max age (recommended 120 seconds on testnet, configurable).
- Do not hardcode feed decimals.
- Record feed ID/value/decimals/timestamp in Quote.
- For USDT0, assume target dollar value only for MVP display; production should add a USDT/USD risk/depeg policy.

14.5 Route capability and liquidity

- At startup and every 60 seconds, check router bytecode, token bytecode, pool address, and pool liquidity.
- USDT0 checkout is disabled if route health fails.
- Set Coston2 demo maximum invoice conservatively (for example 1-2 USDT0) until actual pool depth is measured.
- Before issuing a quote, obtain an exact-output estimate through the configured quoter if available; otherwise use an operator-configured max and display a limitation.
- If exact-output settlement reverts, no partial payout occurs. The attempt enters recovery diagnostics.
- FXRP settlement is the fallback that does not depend on DEX liquidity.

No fake conversion

Do not implement a database-only XRP-to-USDT0 rate or mint a mock stablecoin. The USDT0 path must execute an actual Coston2 token swap through an allowlisted adapter, or the UI must disable it.

15. Xaman integration

15.1 Integration mode

Use the official backend JS/TS SDK. Keep API key and secret on the server. The public checkout can run in a normal browser with QR/deeplink and can later be packaged as a Xaman xApp. The MVP uses a SignIn payload to identify the account, then a Payment payload.

Source anchors: [S10], [S12]

15.2 SignIn security

- Create TransactionType: SignIn payload with custom identifier linked to PayerSession.
- On webhook/poll response, fetch payload from Xaman API; do not trust browser parameters.
- Verify signed result and obtain the r-address.

- Optionally verify the signed blob independently with the XRPL signature verification package.
- Store Xaman user token encrypted for the short checkout session only unless user consents to future push.
- Bind payer session to invoice slug, network, and browser cookie; expire after 30 minutes.

15.3 Payment payload creation

- Require active payer session and quote.
- Re-read personal account nonce immediately before creating payload. If changed, invalidate quote.
- Build transaction template with exact Destination, Amount, Memos, LastLedgerSequence, and testnet network.
- Do not rely on an Account field in the template; Xaman fills it. After signing, verify it equals the identified account.
- Use return URL containing attempt ID, but restore state from server records because mobile redirect behavior can vary.
- Use custom metadata for attempt ID; never put secrets or full userOp bytes in it.

15.4 Webhook handler algorithm

```

1. Read provider delivery identifier and raw body under size limit.
2. Insert WebhookEvent; on unique conflict return 200.
3. Verify provider signature/header when configured.
4. Fetch authoritative payload by payload UUID using Xaman SDK.
5. Match expected payload kind and internal identifier.
6. If rejected/expired, transition safely.
7. If signed Payment, capture txid and enqueue VALIDATE_XRPL.
8. Return 200 quickly; all slow work runs in worker.
9. Never mark invoice SETTLED here.

```

15.5 Xaman test cases

- SignIn accepted, rejected, and expired.
- Payment accepted, rejected, and expired.
- Account differs from identified account.
- Wrong network selected.
- Duplicate webhook deliveries.
- Webhook arrives before browser callback.
- Browser callback arrives without webhook.
- Payer closes tab and resumes from status URL.
- Payload signed after quote expiry.
- User token revoked; fallback to QR/deeplink.

16. Executor, FDC, and direct mint finalization

16.1 Executor identity and funding

- Use a dedicated Coston2 executor EOA private key stored only in deployment secret manager.
- Fund it with C2FLR from faucet for gas.
- Do not use the deployer/admin key as executor.
- Apply transaction nonce locking so only one worker instance signs per executor account, or use a managed signer service.
- Expose executor C2FLR balance metric and alert before it becomes low.

16.2 XRPL validation checklist

- Transaction exists on configured XRPL Testnet and validated == true.
- TransactionType == Payment and meta.TransactionResult == tesSUCCESS.
- Account equals payer session r-address.

- Destination equals quote directMintAddress.
- Amount equals quote xrplPaymentDrops exactly.
- No DestinationTag.
- Exactly the expected 0xFE memo is present; reject extra conflicting Smart Account memo.
- Transaction hash equals Xaman-reported txid.
- Ledger close time is within accepted quote/payment window.
- Transaction has not already been associated with another attempt.

16.3 FDC client boundary

Implement FdcClient behind an interface and port the proven request/proof utility code from the official flare-viem-starter rather than writing encoding from memory. Development can use Flare's rate-limited testnet verifier; production should operate or contract a production-grade verifier service. FDC rounds typically take roughly 90-180 seconds, so the UI must expect a multi-minute processing stage.

Source anchors: [S7], [S15]

```
interface FdcClient {
  requestXrpPaymentProof(input: {
    xrplTxHash: string;
    expectedSourceAddress: string;
    expectedReceivingAddress: string;
    expectedAmountDrops: bigint;
  }): Promise<{ requestId: string; roundId?: bigint }>;

  getXrpPaymentProof(requestId: string): Promise<
    | { status: "PENDING" }
    | { status: "READY"; proof: XrpPaymentProof }
    | { status: "FAILED"; reason: string }
  >;
}
```

16.4 Flare submission algorithm

1. Load attempt + immutable quote under lock.
2. Confirm attempt status FDC_READY and quote/userOp hash consistency.
3. Resolve current AssetManagerFXRP and compare to quote protocol snapshot policy.
4. Decode userOp bytes and re-compute keccak256; compare memo/userOpHash.
5. Sum Call.value; PayMorph v1 requires zero.
6. Simulate executeDirectMintingWithData(proof, userOpData) from executor.
7. If simulation succeeds, submit with EIP-1559 gas policy.
8. Record tx hash before waiting.
9. Wait configured confirmations.
10. Decode DirectMintingExecuted, UserOperationExecuted, PaymentSettled, RecipientPaid, and refund events.
11. Reconcile exact balances/events against Quote and invoice recipients.
12. Mark SETTLED and create immutable receipt projection.

16.5 Retry taxonomy

Class	Examples	Policy
Transient	RPC timeout, 429, provider unavailable, proof pending	Exponential backoff + jitter.
Replacement-safe	Flare tx dropped/underpriced	Same executor nonce, fee bump, record replacement.
Deterministic invalid	Wrong XRPL amount/memo/account	Terminal XRPL_VALIDATION_MISMATCH.
State changed	Personal-account nonce consumed	Do not submit; diagnose/recovery/new payment.
Contract revert	Expired deadline, pool failure, route disabled	Recovery diagnostics; no blind retry.
Already completed	Event found during retry	Reconstruct receipt and mark settled.

17. Recovery and failure handling

17.1 Principle

Recovery objective

A failure after a validated XRP payment must never be hidden. The app must diagnose whether FXRP was minted, whether the transaction ID was consumed, and whether the official 0xE0 skip-memo recovery path is eligible.

17.2 Failure scenarios

Scenario	User-facing result	System action
Xaman rejected	Payment not sent	Close attempt; allow new quote.
Quote expired before signature	Payment payload invalid/expired	Create a new quote and payload.
XRPL signed but invalid fields	Do not claim PayMorph settlement	Terminal investigation; no FDC submission.
FDC proof pending	Processing	Retry within proof window.
Executor unavailable	Processing delayed	Another worker/reconciler resumes.
Flare execution revert	XRP may be at Core Vault; FXRP not settled	Run recovery eligibility diagnosis.
Flare tx succeeded, DB failed	Payment completed	Indexer reconstructs from chain.
USDT0 pool route fails	Atomic revert	Recovery or future FXRP fallback only if protocol allows a new committed operation.

17.3 Official 0xE0 recovery flow

The official recovery path is for a Smart Account direct-mint payment that is stuck at the Core Vault and whose transaction ID has not been used on-chain. A new XRPL payment carrying the 0xE0 skip memo references the original transaction so FXRP is minted to the personal account without executing the original custom instruction. Recovery produces FXRP, not native XRP.

Source anchors: [S18]

1. Read AssetManager transaction-used state for the original XRPL hash.
2. Confirm no successful finalization/payment settlement exists.
3. Generate recovery memo exactly using official Flare helper.
4. Explain to user that recovery mints FXRP to their personal account and skips merchant settlement.
5. Create Xaman Recovery payload on XRPL Testnet.
6. Track recovery transaction and finalization as a separate linked flow.
7. Mark attempt RECOVERED only after FXRP mint evidence.
8. Provide next-step guidance: retry a new PayMorph invoice payment or redeem FXRP through supported FAssets flow.

17.4 Operator runbook thresholds

Condition	Threshold	Action
Awaiting Xaman signature	Payload expiry	Reject/expire attempt.

Condition	Threshold	Action
XRPL signed not validated	2 minutes testnet	Continue querying; alert after 10 minutes.
FDC requested not ready	5 minutes	Retry; alert after 10 minutes.
Flare submitted no receipt	3 minutes	Check tx/mempool and replacement policy.
XRPL validated, no Flare tx	10 minutes	High-priority operator alert.
Recovery required	Immediate after deterministic diagnosis	Display recovery page and alert operator.

18. Security and threat model

18.1 Assets to protect

- Payer intent and exact committed user operation.
- Executor private key and gas funds.
- Merchant sessions and authorization.
- Xaman API secret and user tokens.
- Database integrity of invoice/quote/attempt mapping.
- Contract roles and fee recipient.
- Route configuration and DEX adapter.
- Public receipt correctness.
- User privacy metadata not already public on-chain.

18.2 Threats and mitigations

Threat	Mitigation
Executor substitutes settlement calls	0xFE memo commits keccak256(userOp); on-chain verifies exact bytes.
Replay same payment	Contract settled[paymentId]; unique XRPL tx; Smart Account nonce.
Webhook spoofing	Verify provider signature and fetch authoritative Xaman payload.
Payer changes account	Verify Payment Account equals prior SignIn account.
Wrong vault/amount/memo	Executor exact-field validation before FDC submission.
Merchant edits invoice after quote	Quote stores immutable invoice snapshot; invoice version/hash included in paymentId.
Front-running settlement	Only committed personal-account calls; paymentId and exact values fixed.
Malicious recipient reentrancy	SafeERC20 + nonReentrant; ERC20 transfers only.
Malicious DEX adapter	Allowlisted adapter, onlyRouter restriction, token/pair/fee checks.
Admin steals user tokens	Router should not hold residual funds; restricted rescue with events/paused state, preferably no core-token rescue in v1.
SQL/job duplicate	Unique constraints, row locks, idempotency keys.
RPC inconsistent data	Use confirmations, verify chain ID, optional secondary read provider.
Price manipulation/stale feed	FTSO timestamp validation, caps, slippage, exact-output limit.
Secret leak in logs	Structured redaction; never log secrets, raw signed blobs, or encrypted tokens.
Quote denial/spam	Rate limits, short payer sessions, invoice caps, optional CAPTCHA only if needed.

18.3 Key management

- Local development uses .env.local ignored by git and testnet-only keys.
- Hosted executor uses a secret manager; key is injected at runtime.
- Separate deployer, contract admin, fee manager, pauser, and executor identities where practical.
- After deployment, transfer admin roles to a testnet multisig or clearly documented operator address.
- Never expose Xaman API secret to Next.js client bundle; mark modules server-only.
- Rotate sessions, webhook secrets, and encryption keys with versioned ciphertext.

18.4 Smart-contract security gate

- [x] forge fmt and forge build clean.
- [x] Unit and fuzz tests pass.
- [x] Coverage target $\geq 90\%$ lines and branches for PayMorph contracts.
- [x] Slither/static analysis with no untriaged high severity.
- [x] Manual checklist of approvals, external calls, rounding, reentrancy, roles, pause, replay.
- [x] Coston2 canary amounts before demo.
- [x] Do not call mainnet production-ready without independent audit.

19. Testing strategy

19.1 Test pyramid

Layer	Tools	Required coverage
Pure/domain unit	Vitest	Amount math, transitions, schemas, IDs, quote expiry, idempotency.
Contract unit/fuzz	Foundry	All validations, payouts, fees, refunds, reentrancy, invariants.
API integration	Vitest + test PostgreSQL	Auth, ownership, CRUD, idempotency, webhook dedupe.
Provider adapters	Recorded fixtures + sandbox/testnet	Xaman payload parsing, XRPL tx parsing, FDC proof decoding.
Worker integration	Test DB + mocked transient providers	Job locks, retries, reconciliation, replacement.
Browser E2E	Playwright	Merchant invoice flow and checkout UI states.
Live testnet smoke	Dedicated script	Real SignIn/payment/FDC/settlement with tiny amount.

19.2 Required end-to-end scenarios

- FXRP single-recipient success.
- FXRP multi-recipient success.
- USDT0 single-recipient success with refund.
- USDT0 multi-recipient success.
- Payer rejects SignIn.
- Payer rejects Payment.
- Quote expires before payment.
- Wrong Xaman account attempts payment.
- Duplicate webhook and duplicate worker delivery.
- Executor restart mid-FDC wait.
- Database restart after Flare transaction broadcast.
- USDT0 route disabled; UI offers FXRP.

- Flare execution failure followed by eligible recovery.
- Receipt reconstruction from events after database projection deletion.

19.3 Test data

```
Seed merchant:
  name: Demo Creator Platform
  Coston2 wallet: from DEMO_MERCHANT_ADDRESS

Invoices:
  1. "Design asset license" - 1 XRP - FXRP - one recipient
  2. "Creator bundle" - $1.00 - USDT0 - 85/10/5 split
  3. Expired invoice
  4. Cancelled invoice

Use tiny values compatible with Coston2 liquidity and faucet balances.
Never include real private keys in seed files.
```

19.4 Live smoke script output

- Resolved protocol addresses and bytecode checks.
- FTSO price and freshness.
- Personal account and nonce.
- Quote components and memo hash.
- Xaman payload URL/QR.
- XRPL tx hash and validated ledger.
- FDC request/proof timing.
- Flare tx hash, gas, events.
- Recipient pre/post balances.
- Receipt URL.
- PASS/FAIL summary with machine-readable JSON artifact.

20. Repository and development environment

20.1 Technology choices

Area	Choice
Runtime	Node.js 24 LTS
Package manager	pnpm workspaces
Monorepo	Turborepo or pnpm workspace scripts; avoid unnecessary orchestration complexity
Web	Next.js App Router + TypeScript strict + React + Tailwind
EVM	viem + wagmi
XRPL	xrpl.js
Xaman	Official JS/TS SDK
Contracts	Solidity + Foundry + OpenZeppelin
Database	PostgreSQL + Prisma
Validation	Zod
Tests	Vitest, Playwright, Foundry
Logging	Pino structured JSON

Area	Choice
Formatting/lint	Prettier, ESLint, solhint/forge fmt

Source anchors: [S13], [S14], [S16]

20.2 Monorepo tree

```

paymorph/
  AGENTS.md
  README.md
  SECURITY.md
  LICENSE
  .env.example
  .gitignore
  package.json
  pnpm-workspace.yaml
  turbo.json
  apps/
    web/
      src/app/                # pages + route handlers
      src/components/
      src/features/
      src/lib/server/
      src/lib/client/
      src/styles/
      public/
      tests/
    executor/
      src/index.ts
      src/jobs/
      src/adapters/
      src/clients/
      src/reconcile/
      tests/
  packages/
    contracts/
      src/PayMorphRouter.sol
      src/SparkDexV3Adapter.sol
      src/interfaces/
      script/
      test/
      foundry.toml
    db/
      prisma/schema.prisma
      src/client.ts
      src/queue.ts
    shared/
      src/schemas/
      src/domain/
      src/amounts/
      src/state-machine/
      src/constants/
      src/abis/
    ui/
      src/components/
  scripts/
    resolve-network.ts
    seed.ts
    live-smoke.ts
    verify-deployment.ts
  docs/
    architecture.md
    api/openapi.yaml
    runbooks/
    submission/
  infra/
    docker-compose.yml
    Dockerfile.web
    Dockerfile.executor
    deploy/
  .github/workflows/
    ci.yml
    deploy-preview.yml
    security.yml

```

20.3 Required root scripts

```
pnpm dev          # web + executor + local dependencies
pnpm build        # all packages/apps
pnpm lint
pnpm typecheck
pnpm test         # unit/integration
pnpm test:e2e
pnpm test:contracts
pnpm test:live    # opt-in real testnet smoke
pnpm db:migrate
pnpm db:seed
pnpm contracts:deploy:coston2
pnpm network:resolve
pnpm verify       # lint + typecheck + tests + build
```

20.4 AGENTS.md requirements

Codex reads repository instructions most reliably when they are committed beside the code. Create AGENTS.md with setup commands, architecture invariants, test commands, forbidden shortcuts, and the “one phase per PR” rule. Codex can inspect, edit, and run repository tools from the project directory; use Git checkpoints before and after each task.

Source anchors: [S19], [S20]

```
# AGENTS.md essentials
- Read docs/architecture.md and docs/api/openapi.yaml before editing.
- Never mock the core XRPL -> FDC -> Coston2 settlement in production code.
- All amounts are bigint/base-unit strings; no floating point.
- Do not hardcode FXRP or AssetManagerFXRP; resolve via registry.
- Payment completion requires PaymentSettled event.
- Run pnpm verify and forge tests before finishing.
- Add tests for every behavior change.
- Do not commit secrets, private keys, payload blobs, or generated .env files.
- Keep changes scoped to the current implementation phase.
```

21. Deployment and configuration

21.1 Local development

1. Install Node 24 LTS, pnpm, Foundry, Docker, and Git.
2. docker compose up -d postgres.
3. Copy .env.example to .env.local and add testnet-only credentials.
4. pnpm install; pnpm db:migrate; pnpm db:seed.
5. Fund deployer/executor/merchant addresses with C2FLR; obtain test FXRP/USDT0 as needed.
6. Deploy contracts to Coston2 and write manifest.
7. Start pnpm dev.
8. Expose local Xaman webhook using a secure tunnel only during development; configure exact callback URL.

21.2 Environment variables

Variable	Meaning
APP_ENV	development preview production
APP_URL	Public HTTPS base URL
DATABASE_URL	PostgreSQL connection

Variable	Meaning
SESSION_SECRET	High-entropy session signing secret
DATA_ENCRYPTION_KEY_V1	32-byte ciphertext key
COSTON2_RPC_URL	Coston2 HTTPS RPC
COSTON2_WSS_URL	Coston2 WSS
COSTON2_CHAIN_ID	114
XRPL_WS_URL	XRPL Testnet WSS
XRPL_NETWORK	testnet
XAMAN_API_KEY	Server-only
XAMAN_API_SECRET	Server-only
XAMAN_WEBHOOK_SECRET	If supported/configured
EXECUTOR_PRIVATE_KEY	Server-only Coston2 test key
FLARE_CONTRACT_REGISTRY	Registry address
USDT0_ADDRESS	Coston2 USDT0
SPARKDEX_ROUTER_ADDRESS	Configured official router
SPARKDEX_FACTORY_ADDRESS	Required for pool checks
SPARKDEX_QUOTER_ADDRESS	Optional but recommended exact-output quote
SPARKDEX_POOL_FEE	500
PAYMORPH_ROUTER_ADDRESS	Deployment output
PAYMORPH_ADAPTER_ADDRESS	Deployment output
SERVICE_FEE_BPS	50; must match contract
MAX_QUOTE_TTL_SECONDS	900
MAX_FTSO_AGE_SECONDS	120
DEFAULT_SLIPPAGE_BPS	150
MAX_USDT0_DEMO_BASE_UNITS	Environment cap
EXECUTOR_FEE_UBA	Configured and protocol-valid
FDC_VERIFIER_URL	Testnet verifier
FDC_DA_LAYER_URL	Official/testnet proof service used by starter
LOG_LEVEL	info
SENTRY_DSN	Optional
ADMIN_WALLET_ADDRESSES	Comma-separated checksummed addresses

21.3 Startup readiness checks

- Database reachable and migrations current.
- Coston2 RPC chain ID == 114.
- Registry resolves AssetManagerFXRP and FtsoV2.
- FXRP/USDT0/router/adaptor/PayMorphRouter bytecode present.
- Contract service fee and fee recipient match expected config.
- XRPL Testnet server connected.
- Xaman credentials can create/delete a test payload in non-production or pass API auth check.
- Executor account has minimum C2FLR.

- FTSO feed fresh.
- USDT0 route health determined; route may be degraded while FXRP remains ready.

21.4 Hosted deployment

- Web: Vercel or container host with Node runtime capable of server routes and long-lived SSE; use polling fallback if serverless limits apply.
- Executor: always-on container/worker host, not a short-lived serverless function.
- Database: managed PostgreSQL with backups and connection pooling.
- Secrets: host secret manager; no values in build logs.
- HTTPS mandatory for Xaman callbacks and cookies.
- Run database migration as a release step before traffic switch.
- Deploy contracts before web release and commit the deployment manifest.

22. Observability and operations

22.1 Correlation fields

```
Every structured log should include when available:
requestId, merchantId, invoiceId, payerSessionId, quoteId,
attemptId, paymentId, xamanPayloadUuid, xrplTxHash,
flareTxHash, userOpHash, jobId, jobType, network.
```

22.2 Metrics

Metric	Use
paymorph_quotes_total	By asset, denomination, result.
paymorph_attempts_total	By terminal status.
paymorph_step_duration_seconds	Histogram by transition.
paymorph_fdc_wait_seconds	Proof latency.
paymorph_settlement_seconds	XRPL validation to PaymentSettled.
paymorph_executor_jobs_ready	Queue depth.
paymorph_executor_job_retries_total	By error code.
paymorph_executor_c2flr_balance	Gas balance.
paymorph_usdt0_route_healthy	0/1 capability.
paymorph_ftso_age_seconds	Feed freshness.
paymorph_recovery_required_total	Safety KPI.
paymorph_webhook_duplicates_total	Provider retry/dedupe health.

22.3 Alerts

- Executor C2FLR below threshold.
- No worker heartbeat for 2 minutes.
- XRPL validated attempt without Flare submission for 10 minutes.
- Flare transaction failures above threshold.
- USDT0 route unhealthy for more than 5 minutes.
- FTSO stale.
- Xaman webhook error rate elevated.

- Database connection or migration mismatch.
- Any RECOVERY_REQUIRED attempt.
- Contract paused or config mismatch.

22.4 Operator dashboard

- System health cards.
- Pending attempts by state and age.
- Attempt detail with immutable snapshots and all hashes.
- Safe retry buttons only for retryable jobs.
- Recovery eligibility diagnostics.
- Route disable/enable at application layer.
- Contract pause state and links; contract actions require wallet transaction, not server secret.
- Audit trail of operator actions.

23. CI/CD and release gates

23.1 Pull-request CI

- [x] Install pinned dependencies with frozen lockfile.
- [x] Lint and formatting check.
- [x] TypeScript typecheck.
- [x] Unit/integration tests with PostgreSQL service.
- [x] Foundry build/test/fuzz.
- [x] Playwright smoke with mocked provider boundaries only for UI behavior.
- [x] Build all apps.
- [x] Secret scan and dependency audit.
- [x] Generate OpenAPI validation and ensure route/schema drift fails CI.

23.2 Coston2 release pipeline

1. Deploy/verify contracts only from tagged release or explicit manual workflow.
2. Write addresses, chain ID, commit SHA, constructor args, and tx hashes to deployment manifest.
3. Run verify-deployment script.
4. Run database migrations.
5. Deploy executor, then web.
6. Run live smoke FXRP path.
7. Run live smoke USDT0 path if route healthy.
8. Publish receipt URLs and artifact JSON in release summary.
9. Rollback app release on failure; contracts are immutable and versioned.

23.3 Release blockers

- Any failing core contract test.
- Any unresolved high-severity static-analysis finding.
- Hardcoded FXRP/AssetManager address.
- Core flow uses a mock/fake success path.
- Receipt can be marked paid without PaymentSettled.
- No recovery diagnostics.
- Secrets detected.
- Live FXRP smoke fails.
- Wrong/testnet disclaimer missing.

24. Product analytics and privacy

24.1 Minimal analytics events

Event	Properties
invoice_created	denomination, settlementAsset, recipientCount
invoice_published	invoiceId
checkout_viewed	invoiceId, deviceClass
xaman_signin_started	invoiceId
xaman_signin_completed	durationMs
quote_created	asset, route, amountBucket, warnings
payment_signature_started	asset
xrpl_validated	durationMs
fdc_ready	durationMs
payment_settled	asset, recipientCount, totalDurationMs
payment_failed	step, errorCode, recoverable
recovery_started	failureCode
recovery_completed	durationMs

24.2 Privacy rules

- Do not send wallet addresses, transaction hashes, invoice descriptions, or exact amounts to third-party analytics.
- Use coarse amount buckets if analytics is enabled.
- Hash IP addresses with rotating salt for abuse control; do not retain raw IP unless operationally required and disclosed.
- Clearly disclose that blockchain addresses and transactions are public.
- Do not place private invoice details in XRPL memo or on-chain event fields; use paymentId/content hash only.
- Encrypt user tokens and userOp bytes; delete expired payer-session tokens on schedule.
- Provide merchant data deletion for off-chain profile data while preserving legally/operationally necessary on-chain references.

25. Demo script and judge experience

25.1 Recommended 2-minute live demo

1. Open merchant dashboard and show deployed Coston2 contract address.
2. Create a \$1.00 Creator Bundle invoice with Creator 85%, Affiliate 10%, Platform 5%, settlement USDT0.
3. Open public checkout in a fresh browser/mobile view.
4. Connect Xaman using SignIn.
5. Show quote: customer pays test XRP; merchant side receives exact test USDT0; show fee and quote timer.
6. Sign one XRP Testnet payment.
7. Show live timeline: XRPL validated -> FDC proof -> Coston2 settlement.
8. Open XRPL explorer transaction.
9. Open Coston2 transaction and PayMorph events.
10. Show receipt with three USDT0 payouts and any FXRP refund.

11. State: all tokens are testnet and valueless; no core step was mocked.

25.2 Backup demo

- Pre-record the complete live flow, including explorer pages.
- Keep a pre-settled receipt ready.
- Keep an FXRP invoice ready in case testnet DEX liquidity is temporarily unavailable.
- Have a network diagnostics page proving current addresses and route health.
- Do not fake a fresh transaction; clearly label fallback recording and show verifiable hashes.

25.3 Judge-facing explanation

Pitch

PayMorph lets an XRP user pay a normal merchant invoice without bridging manually. The merchant chooses FXRP or stable USDT0 settlement, and Flare Smart Accounts atomically execute the conversion, fee, and revenue split from one XRPL payment.

26. Hackathon submission package

26.1 Required submission fields

Field	Content
Project name	PayMorph
Bounty	Bounty 1 - Interoperable Asset Products
Description	Native XRP checkout with merchant-selected FXRP/USDT0 settlement and optional automatic splits.
Target user	Digital merchants, freelancers, creator platforms, small marketplaces.
Demo	Public hosted app + 2-minute video + testnet instructions.
Repository	Public GitHub monorepo with licenses, setup, architecture, tests.
Flare usage	FAssets/FXRP, Smart Accounts 0xFE, FDC XRPPayment proof, FTSO, Coston2 contracts.
New work	PayMorph contracts, Xaman checkout, executor, quote engine, receipts, recovery UX.
Deployment	XRPL Testnet + Coston2; list router/adaptor addresses and tx hashes.
Roadmap	Security review, mainnet route, API/SDK, pilots, off-ramp partners.

26.2 Repository documentation checklist

- **[x]** README with one-sentence pitch and architecture diagram.
- **[x]** Exact testnet setup and faucet links.
- **[x]** Live app and demo video.
- **[x]** Contract addresses and explorers.
- **[x]** Security assumptions and limitations.
- **[x]** Recovery behavior.
- **[x]** API reference.
- **[x]** Testing commands and live-smoke output.
- **[x]** What was built during hackathon.
- **[x]** Roadmap and PMF validation plan.

- **[x]** License and contribution instructions.

26.3 Evidence of seriousness

- Five interviews with XRP-holding freelancers/merchants.
- At least three users complete a testnet checkout.
- Record time-to-complete and confusion points.
- One creator/marketplace pilot letter or message.
- Public issue tracker and roadmap.
- Telemetry showing successful end-to-end attempts, with privacy-safe aggregation.

27. Mainnet roadmap and PMF path

27.1 Beachhead market

Start with digital-service and creator merchants who already have XRP-native customers. They avoid physical-goods disputes, can deliver globally, and often have collaborators or affiliates who need automatic splits.

27.2 30/60/90-day roadmap

Window	Goals
0-30 days	Complete Coston2 product; 5-10 user tests; tighten checkout; measure route reliability; contract review.
31-60 days	External security review; mainnet configuration rehearsal; merchant API/webhooks; reusable payout templates; 3 pilot merchants.
61-90 days	Controlled mainnet beta with caps; monitoring/support; branded checkout; stable settlement/liquidity partner; legal review.
Beyond	Additional FAssets, settlement adapters, subscriptions, platform SDK, local off-ramp partnerships where compliant.

27.3 PMF hypotheses to test

- XRP holders want to spend XRP without first moving into another wallet/network.
- Merchants will accept XRP if they can avoid holding volatile XRP.
- A visibly simpler one-signature flow materially improves conversion.
- Automatic revenue splits reduce manual accounting for creator/marketplace transactions.
- A public cross-chain receipt increases trust enough to compensate for multi-minute settlement.

27.4 Metrics

- Checkout start-to-sign conversion.
- Sign-to-settlement success rate.
- Median settlement time.
- Quote refresh/expiry rate.
- USDT0 versus FXRP settlement selection.
- Repeat merchant invoice creation.
- Support incidents per 100 attempts.
- Recovery-required rate.
- Merchant pilot retention and willingness to pay.

27.5 Mainnet prerequisites

- Independent smart-contract audit and remediation.

- Production-grade FDC/executor operations and redundancy.
- Mainnet FXRP/USDT0 route liquidity and slippage policy.
- Key management/HSM or managed signer.
- Legal/compliance review for target jurisdictions and business model.
- Terms, privacy, support, incident response, and monitoring.
- Controlled limits and emergency pause.
- Real user testing with explicit risk disclosures.

28. Codex implementation plan

Each phase should be a separate branch/PR. Codex must read the previous phase output, run all relevant tests, and leave a short completion report. Do not advance if acceptance criteria fail.

Phase	Build	Exit gate
Phase 0 - Repository foundation	Create workspace, strict TS, lint/format, Docker Postgres, Prisma, shared response/error libraries, CI skeleton.	pnpm verify passes; health route; DB migration; no secrets.
Phase 1 - Contracts	Implement router, adapter, interfaces, deployment scripts, unit/fuzz/invariant tests.	All Foundry tests; deployment manifest schema; no residual token invariant.
Phase 2 - Network package	Registry discovery, FXRP/FTSO/FAssets helpers, amount utilities, address capability checks.	resolve-network script prints verified Coston2 config; unit fixtures.
Phase 3 - Merchant auth and invoices	Wallet challenge auth, session, merchant CRUD/profile, invoice schema/routes/dashboard.	Merchant can create/publish/cancel invoice; ownership tests.
Phase 4 - Public checkout and Xaman SignIn	Public invoice route/page, payer session, SignIn payload, webhook ingestion, account binding.	SignIn accepted/rejected/duplicate webhook tests; mobile/desktop UI.
Phase 5 - Quote and userOp	FTSO/pricing, personal account/nonce, Call[] encoding, 0xFE memo, immutable Quote.	Golden-vector tests compare hash/bytes; no floats; expiry/nonce tests.
Phase 6 - Xaman Payment and XRPL validation	Create Payment payload; status page; worker job; exact transaction validation.	Real or fixture-signed testnet payload validates; mismatches fail terminally.
Phase 7 - FDC executor	Port official FDC helpers; proof lifecycle; finalization simulation/submission; event decoding.	Real tiny FXRP settlement on Coston2; restart-safe job state.
Phase 8 - USDT0 route	SparkDEX adapter deployment/config, route health, exact-output quote/settlement/refund.	Real tiny USDT0 settlement or route disabled with explicit reason.
Phase 9 - Receipts and reconciliation	Public receipt, event projection, reconciler, CSV export.	Delete projection and reconstruct from chain; exact balance checks.
Phase 10 - Recovery	Eligibility diagnostics and 0xE0 Xaman flow.	Official recovery scenario tested on Coston2 or documented reproducible script.
Phase 11 - Hardening	Rate limits, CSRF, secret redaction, admin ops, metrics, alerts, a11y, error UX.	Security checklist and all test suites pass.
Phase 12 - Deployment and submission	Hosted deploy, live smoke, docs, video script, submission files.	Judge can execute full testnet flow from README.

28.1 Codex task template for each phase

Goal: Implement Phase <N> exactly as specified in PayMorph Blueprint.

Before editing:

1. Read AGENTS.md, docs/architecture.md, docs/api/openapi.yaml.
2. Inspect existing code and tests.
3. State the files you expect to change.

Rules:

```

- Do not change product/protocol invariants without documenting a blocker.
- Do not mock core chain behavior in production paths.
- Add or update tests first/with implementation.
- Use bigint/base-unit strings only.
- Preserve API envelopes and state-machine guards.

Verification:
- Run the phase-specific commands.
- Run pnpm verify.
- Report commands, results, changed files, remaining risks.
- Stop if a required official protocol detail cannot be verified; leave a typed adapter/TODO
  and cite the exact official source needed rather than inventing behavior.

```

28.2 Implementation order within a phase

1. Write/update schemas and domain interfaces.
2. Write failing unit/contract tests.
3. Implement pure/domain logic.
4. Implement provider adapter behind interface.
5. Implement API/worker integration.
6. Implement UI.
7. Run tests and static analysis.
8. Update docs and changelog.
9. Commit a checkpoint.

29. Definition of done

29.1 Product definition of done

- ☐ Merchant can sign in with a Coston2 wallet.
- ☐ Merchant can create/publish a valid invoice with 1-10 recipients.
- ☐ Customer can identify with Xaman.
- ☐ Customer receives a quote built from current protocol/FTSO data.
- ☐ Customer signs exactly one XRP Testnet payment.
- ☐ Payment validates on XRPL Testnet.
- ☐ FDC proof and executeDirectMintingWithData are real.
- ☐ PayMorphRouter emits settlement and recipient events.
- ☐ FXRP path works end to end.
- ☐ USDT0 path works end to end or is automatically disabled with honest UX.
- ☐ Receipt links both chains and reconciles exact amounts.
- ☐ Duplicate delivery cannot duplicate settlement.
- ☐ A stuck direct mint has a recovery diagnosis and user path.
- ☐ Hosted app and public repo are usable by judges.

29.2 Engineering definition of done

- ☐ No TypeScript errors, lint failures, or formatting drift.
- ☐ No failing unit/integration/e2e/contract tests.
- ☐ No floating-point monetary code.
- ☐ No hardcoded dynamic Flare protocol addresses.
- ☐ No secrets or keys committed.
- ☐ All API mutations validated and authorized.
- ☐ All state changes use transition guard.
- ☐ All jobs idempotent/restart-safe.

- ☐ All contract external calls guarded and tested.
- ☐ OpenAPI and implementation synchronized.
- ☐ README reproduces setup.
- ☐ Live smoke artifact archived.
- ☐ PDF blueprint deviations documented in ADRs.

29.3 Judge acceptance checklist

- ☐ Can explain product in one sentence.
- ☐ Can identify target user and pain.
- ☐ Can show a real public app.
- ☐ Can sign a real XRP Testnet transaction.
- ☐ Can show FXRP mint/settlement on Coston2.
- ☐ Can show Flare-specific integration clearly.
- ☐ Can verify contract addresses and GitHub.
- ☐ Can explain what was newly built.
- ☐ Can show error/recovery behavior.
- ☐ Can explain credible mainnet path and user validation.

30. Master Codex prompt

Use this prompt after creating the repository

Paste the following into Codex with this PDF available in the project. Ask Codex to implement only Phase 0 first. After review, continue one phase at a time.

```
You are the lead engineer for PayMorph, a Flare Summer Signal Bounty 1 product.
The authoritative product and engineering specification is PayMorph_Blueprint.pdf.
Read it fully before changing files.

Objective
Build a complete, testable product where:
1. A merchant creates an XRP- or USD-denominated invoice.
2. The merchant chooses FXRP or USDT0 settlement and optional recipient splits.
3. A payer identifies their XRPL Testnet account using Xaman SignIn.
4. The system creates a 0xFE Flare Smart Account custom instruction.
5. The payer signs one real XRP Testnet Payment to the current FAssets direct-mint address.
6. An executor obtains the FDC XRPPayment proof and calls executeDirectMintingWithData.
7. The personal account atomically approves FXRP and calls PayMorphRouter.
8. PayMorphRouter distributes FXRP or swaps exact output to USDT0 through the allowlisted adapter.
9. The app shows a verifiable receipt with XRPL and Coston2 explorer links.

Non-negotiable invariants
- No fake success path for XRPL, FDC, FXRP minting, Flare execution, swap, or settlement.
- No JavaScript floating point for monetary values; use bigint/base-unit strings.
- Resolve FXRP and AssetManagerFXRP at runtime through the Flare registry/protocol.
- Xaman SignIn occurs before constructing the payer-specific personal account and nonce.
- Use the 0xFE hash-commitment flow and store exact immutable userOp bytes.
- The Xaman webhook is only a notification; fetch authoritative payload details.
- Validate XRPL Account, Destination, Amount, memo, no DestinationTag, network, validation, and tesSUCCESS.
- Payment completion requires PayMorphRouter PaymentSettled event.
- Every API mutation and job is idempotent.
- On-chain settled[paymentId] prevents replay.
- USDT0 route must be real and health-checked; otherwise disable it and keep FXRP available.
- Implement official 0xE0 recovery diagnostics/path for stuck direct mints.
- Never commit secrets or private keys.

Execution process
- Implement one numbered phase from Section 28 at a time.
- Start with Phase 0 only.
- Before coding, create AGENTS.md from the blueprint and write a short phase plan.
- Add tests with every implementation.
- Run all phase exit-gate commands and pnpm verify.
- Do not silently change the architecture. If an official protocol detail differs from the PDF,
  cite the official source in an ADR and update the implementation/spec accordingly.
```

- Do not invent contract addresses or FDC encoding. Port/adapt the official Flare starter utilities.
- At the end, provide: changed files, commands run, test results, known risks, and next phase.

Begin by reading the PDF and implementing Phase 0 - Repository foundation.

Appendix A. Canonical domain rules

Rule	Definition
PAYMENT_ID	keccak256(chainDomain, invoiceId, quoteId, payerXrplAccount, userOpHash); generated once and stored.
INVOICE_MUTABILITY	Draft editable. Published invoice financial fields immutable; cancel/expire only. Create a replacement invoice for changes.
QUOTE_MUTABILITY	Immutable and single-use.
PAYER_BINDING	SignIn r-address must equal Payment Account.
NONCE_BINDING	Quote uses current personal-account nonce; re-check before Payment payload.
SETTLEMENT_SOURCE	Only PayMorphRouter events establish application settlement.
RECIPIENT_SUM	Exactly 10,000 bps.
MAX_RECIPIENTS	10 in v1.
FEE_MODEL	Payer funds invoice amount plus PayMorph service fee.
REFUND	USDT0 exact-output unused FXRP returns to payer personal account/refundTo.
EXPIRY	Quote and contract deadline aligned; payment payload never outlives quote.
NETWORK	MVP accepts XRPL Testnet and Coston2 only.
TESTNET_LABEL	All user-facing financial screens state tokens have no real value.

Appendix B. API response projections

```

PublicInvoice {
  slug, title, description?, externalRef?, merchant: {displayName, logoUrl?},
  denomination, amountDisplay, settlementAsset, expiresAt,
  recipients: [{label, addressMasked, bps}], status,
  network: {source:"XRPL_TESTNET", destination:"COSTON2"}
}

Receipt {
  attemptId, paymentId, status:"SETTLED", testnet:true,
  invoice: {...}, payer: {xrplAccount},
  sourcePayment: {xrpDrops, xrpDisplay, txHash, explorerUrl, validatedAt},
  settlement: {asset, invoiceAmount, serviceFee, inputFxrUsed, refundFxr,
    flareTxHash, explorerUrl, settledAt, routerAddress, version},
  recipients: [{label,address,bps,amount,token,txHash}],
  protocol: {userOpHash, personalAccount, assetManagerAddress}
}

```

Appendix C. Domain service interfaces

```

interface QuoteService {
  createQuote(input: {
    invoiceId: string;
    payerXrplAccount: string;
    slippageBps: number;
  }): Promise<Quote>;
}

interface SmartAccountEncoder {

```



```

derivePersonalAccount(xrplAccount: string): Promise<Address>;
getNonce(personalAccount: Address): Promise<bigint>;
encodePaymentOperation(input: SettlementPlan): Promise<{
  calls: Call[];
  packedUserOpData: Hex;
  userOpHash: Hex;
  memoHex: Hex;
}>;
}

interface XamanGateway {
  createSignIn(input: SignInRequest): Promise<XamanPayload>;
  createPayment(input: PaymentTemplate): Promise<XamanPayload>;
  getPayload(uuid: string): Promise<XamanPayloadResult>;
}

interface XrplGateway {
  getValidatedPayment(txHash: string): Promise<ValidatedXrplPayment | null>;
}

interface FlareFinalizer {
  simulate(proof: XrpPaymentProof, userOpData: Hex): Promise<void>;
  submit(proof: XrpPaymentProof, userOpData: Hex): Promise<Hash>;
  decodeSettlement(receipt: TransactionReceipt): Promise<SettlementEvidence>;
}

```

Appendix D. Payment timeline copy

Internal state	Customer-facing copy
IDENTIFYING	Confirm your XRP account in Xaman.
QUOTED	Your XRP amount is locked until {time}.
AWAITING_SIGNATURE	Review and approve the payment in Xaman.
XRPL_SIGNED	Payment signed. Waiting for XRP Ledger validation.
XRPL_VALIDATED	XRP payment confirmed.
FDC_REQUESTED	Flare is verifying the XRP payment.
FDC_READY	Payment verified. Preparing settlement on Flare.
FLARE_SUBMITTED	Settlement transaction submitted on Coston2.
SETTLED	Payment complete. The merchant has been settled.
RECOVERY_REQUIRED	The XRP payment was confirmed, but the Flare action did not finish. Your value can be recovered as FXRP.

Appendix E. Demo seed and operator checklist

- ☐ Verify Coston2 and XRPL Testnet status.
- ☐ Verify executor C2FLR balance.
- ☐ Resolve and record protocol addresses.
- ☐ Verify contracts unpaused.
- ☐ Verify USDT0 pool route and amount cap.
- ☐ Run FXRP live smoke.
- ☐ Run USDT0 live smoke if enabled.
- ☐ Create demo invoice and backup receipt.
- ☐ Open Xaman on testnet account with sufficient test XRP.
- ☐ Open explorer tabs.

- ☐ Start screen recording.
- ☐ Keep recovery demo attempt available.

Appendix F. OpenAPI implementation notes

- Commit docs/api/openapi.yaml and generate types from it or validate Zod schemas against it.
- Every path documents auth, Idempotency-Key, rate limits, error codes, and examples.
- Use cursor pagination: {items, nextCursor}; never offset for mutable attempt lists.
- Dates are RFC3339 UTC strings.
- Amounts are decimal strings with asset/baseUnits/display, never JSON numbers.
- Public endpoints redact internal errors and encrypted/provider fields.
- SSE sends status snapshot then transition events; heartbeat every 15 seconds; polling endpoint remains canonical.

Appendix G. Security review questions

- ☐ Can any actor change recipients after the memo hash is signed?
- ☐ Can a duplicate webhook or job submit a second settlement?
- ☐ Can the contract leave FXRP/USDT0 dust?
- ☐ Can an adapter call back into router?
- ☐ Can admin redirect a pending user payment?
- ☐ What happens if the fee changes after quote?
- ☐ What happens if the personal-account nonce changes?
- ☐ What happens if FTSO is stale?
- ☐ What happens if exact-output swap needs more than max input?
- ☐ What happens if Flare tx succeeds but receipt processing crashes?
- ☐ How is executor nonce managed across replicas?
- ☐ Can logs or analytics leak invoice details or user tokens?
- ☐ Can recovery be triggered after a successful mint?
- ☐ Are all dynamic addresses resolved and verified on the expected chain?
- ☐ Does every user-facing “paid” state have PaymentSettled evidence?

Appendix H. Source anchors and official references

- [S1] Flare Summer Signal hackathon details and judging criteria: <https://dorahacks.io/hackathon/flaresummersignal/detail>
- [S2] Flare network overview: Coston2 RPC, chain ID, explorer, faucet: <https://dev.flare.network/network/overview>
- [S3] Flare Smart Accounts overview: <https://dev.flare.network/smart-accounts/overview>
- [S4] 0xFE Custom Instruction protocol and memo layout: <https://dev.flare.network/smart-accounts/custom-instruction>
- [S5] Custom Instruction TypeScript/Viem guide: <https://dev.flare.network/smart-accounts/guides/typescript-viem/custom-instruction-ts>
- [S6] FAssets minting and executeDirectMintingWithData: <https://dev.flare.network/fassets/minting>
- [S7] FDC getting started, verifier, DA layer, round timing: <https://dev.flare.network/fdc/getting-started>
- [S8] Read FAssets settings and XRP/USD feed on Coston2: <https://dev.flare.network/fassets/developer-guides/fassets-settings-node>
- [S9] Smart Account USDT0/SparkDEX Coston2 guide: <https://dev.flare.network/smart-accounts/guides/typescript-viem/control-usdt0-ts>
- [S10] Xaman SignIn, payloads, webhooks, authorization: <https://docs.xaman.dev/concepts/special-transaction-types/signin>
- [S11] XRPL send XRP and testnet faucet: <https://xrpl.org/docs/tutorials/how-tos/send-xrp>
- [S12] Xaman payload lifecycle and account filling: <https://docs.xaman.dev/concepts/payloads-sign-requests>
- [S13] Next.js App Router documentation: <https://nextjs.org/docs/app>
- [S14] Viem documentation: <https://viem.sh/docs/getting-started>
- [S15] Official Flare Viem Smart Accounts starter: <https://github.com/flare-foundation/flare-viem-starter>
- [S16] Official Flare FAssets demo dApp: <https://github.com/flare-foundation/fassets-demo-dapp>
- [S17] OpenAI Codex CLI official documentation: <https://developers.openai.com/codex/cli>

[S18] Official stuck direct-mint recovery guide: <https://dev.flare.network/smart-accounts/guides/typescript-viem/recover-stuck-mint-transaction-ts>

[S19] OpenAI Codex best practices: <https://developers.openai.com/codex/learn/best-practices>

[S20] AGENTS.md open format: <https://agents.md/>

Appendix I. Final caveats

- Protocol contracts, addresses, and SDKs can change. Runtime discovery and official-package pinning are mandatory.
- Coston2 and XRPL Testnet can reset or experience outages. Store enough evidence to rerun the demo.
- USDT0 route availability and liquidity must be measured at implementation time.
- The blueprint does not replace a smart-contract audit or legal/compliance review.
- A mainnet product requires limits, operations, monitoring, support, security review, and jurisdiction-specific analysis.
- When official Flare documentation conflicts with this document, use the current official documentation, record the change in an Architecture Decision Record, and update tests/spec.